

CPIFP Alan Turing

Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Web (DAW)

MEMORIA DEL TRABAJO FIN DE GRADO

TechUniverse

Plataforma de comercio electrónico tecnológico: desarrollo full-stack con Angular, React y Laravel, y despliegue automatizado en la nube (AWS)

Autores:

Iván Ríos Raya · Alexander Sánchez Jara

Curso académico 2025 / 2026

Málaga, mayo de 2026

Resumen

TechUniverse es una plataforma de comercio electrónico orientada a la venta de productos tecnológicos (smartphones, ordenadores portátiles, audio y accesorios) desarrollada como Trabajo Fin de Grado del Ciclo Formativo de Grado Superior de Desarrollo de Aplicaciones Web. El proyecto aborda el ciclo completo de una aplicación web moderna: una **tienda pública** construida como aplicación de página única (SPA) en **Angular 21**, enriquecida con dos componentes desarrollados en **React 19** y embebidos como *Web Components*; un **servicio web REST** implementado con **Laravel 12** que expone el catálogo, las categorías, las especificaciones técnicas, el carrito de la compra, la lista de deseos y la gestión de usuarios, protegido mediante autenticación por *tokens* con **Laravel Sanctum** y documentado con **OpenAPI (L5-Swagger)**; un **panel de administración** para la gestión del catálogo y de los usuarios; y una **infraestructura de despliegue automatizado** en **Amazon Web Services**, definida como código con **Terraform** y orquestada mediante **GitHub Actions** y **AWS CodeDeploy**.

La memoria documenta el contexto y los objetivos del proyecto, la planificación y la metodología de trabajo del equipo, el estado del arte y la justificación de las tecnologías elegidas, el análisis de requisitos, el diseño del sistema (arquitectura, modelo de datos, API, frontend, interfaz de usuario, panel de administración, infraestructura y seguridad), la implementación de cada subsistema, la estrategia y los resultados de las pruebas, y unas conclusiones que incluyen el grado de consecución de los objetivos, las dificultades encontradas, la deuda técnica identificada y las líneas de trabajo futuro. Se ha optado por una arquitectura desacoplada cliente-servidor, por componentes *standalone* y *signals* en el frontend, por un ORM expresivo y *API Resources* en el backend, y por una estrategia de *DevOps* basada en infraestructura como código, con el fin de obtener una solución mantenible, escalable y representativa de las prácticas profesionales actuales del desarrollo web *full-stack*.

Palabras clave: comercio electrónico, aplicación de página única, API REST, Angular, React, Web Components, Laravel, Sanctum, OpenAPI, ORM, Tailwind CSS, infraestructura como código, Terraform, AWS, CodeDeploy, integración continua, despliegue continuo.

Índice

Resumen.....	1
Índice.....	2
1. Introducción.....	5
1.1. Contexto y motivación.....	5
1.2. Descripción general del proyecto.....	5
1.3. Objetivos.....	6
1.3.1. Objetivo general.....	6
1.3.2. Objetivos específicos.....	6
2. Planificación y metodología.....	7
2.1. Equipo de trabajo y reparto de tareas.....	7
2.2. Gestión del código fuente: estrategia de ramas.....	7
2.3. Herramientas de gestión, diseño y desarrollo.....	8
2.4. Cronograma y fases del proyecto.....	8
3. Tecnologías y estado del arte.....	10
3.1. Arquitectura de referencia: SPA + API REST.....	10
3.2. Tecnologías del backend.....	10
3.2.1. PHP y Laravel 12.....	10
3.2.2. L5-Swagger / OpenAPI.....	11
3.2.3. Panel de administración: Blade.....	11
3.2.4. PHPUnit.....	11
3.3. Tecnologías del frontend.....	11
3.3.1. Angular 21.....	11
3.3.2. Tailwind CSS y sistema de diseño.....	12
3.3.3. Lucide y otras librerías de UI.....	12
3.3.4. Vitest.....	12
3.4. React y Web Components: interoperabilidad de frameworks.....	12
3.5. Almacenamiento de objetos: Cloudflare R2 / Amazon S3.....	13
3.6. Infraestructura y despliegue.....	13
3.6.1. Amazon Web Services (EC2, VPC, Route 53).....	13
3.6.2. Terraform (infraestructura como código).....	13
3.6.3. GitHub Actions (integración y entrega continuas).....	14
3.6.4. AWS CodeDeploy.....	14
3.6.5. Apache, Certbot/Let's Encrypt y DuckDNS.....	14
3.7. Justificación de la pila tecnológica.....	15
4. Análisis del sistema.....	16
4.1. Identificación de actores.....	16
4.2. Casos de uso.....	16
4.3. Reglas de negocio.....	17
5. Diseño del sistema.....	19
5.1. Arquitectura general.....	19

5.2. Modelo Entidad/Relación.....	20
5.3. Diseño de la API REST.....	21
5.4. Diseño del frontend.....	21
5.5. Diseño de la interfaz de usuario.....	22
5.6. Diseño del panel de administración.....	23
5.7. Diseño de la infraestructura de despliegue.....	23
5.8. Diseño de la seguridad.....	24
6. Implementación.....	25
6.1. Organización del repositorio.....	25
6.2. Implementación del backend (API REST con Laravel).....	25
6.2.1. Estructura del proyecto Laravel.....	25
6.2.2. Modelos Eloquent y relaciones.....	26
6.2.3. Migraciones y poblado de datos.....	27
6.2.4. Controladores de la API y recursos.....	27
6.2.5. Autenticación y autorización.....	28
6.2.6. Almacenamiento de imágenes en R2.....	28
6.2.7. Documentación de la API con OpenAPI.....	29
6.3. Implementación del panel de administración.....	29
6.4. Implementación del frontend (Angular).....	30
6.4.1. Arranque y configuración.....	30
6.4.2. Enrutado y carga diferida.....	30
6.4.3. Componentes reutilizables.....	30
6.4.4. Páginas.....	31
6.4.5. Servicios y gestión de estado con señales.....	31
6.4.6. Consumo de la API e interceptor de autenticación.....	32
6.5. Implementación de los componentes React (Web Components).....	33
6.5.1. El subproyecto React-components.....	33
6.5.2. Empaquetado y registro como Custom Elements.....	33
6.5.3. El asistente conversacional (chatbot).....	33
6.6. Implementación de la infraestructura y el despliegue.....	34
6.6.1. Infraestructura como código con Terraform.....	34
6.6.2. Aprovisionamiento de los servidores (user-data).....	34
6.6.3. Integración y entrega continuas con GitHub Actions.....	35
6.6.4. Despliegue con AWS CodeDeploy.....	36
7. Pruebas.....	37
7.1. Estrategia de pruebas.....	37
7.2. Pruebas del backend.....	37
7.3. Pruebas del frontend.....	38
7.4. Pruebas manuales y de aceptación.....	38
7.5. Integración continua.....	39
8. Conclusiones y líneas futuras.....	40
8.1. Grado de consecución de los objetivos.....	40

8.2. Dificultades encontradas y lecciones aprendidas.....	40
8.3. Valoración final.....	41

(Si el índice aparece vacío o desactualizado: haz clic con el botón derecho sobre él y elige «Actualizar campos» → «Actualizar toda la tabla».)

1. Introducción

1.1. Contexto y motivación

El comercio electrónico se ha consolidado como uno de los canales de venta principales del sector tecnológico. Los consumidores esperan tiendas en línea rápidas, accesibles desde cualquier dispositivo, con catálogos extensos, búsqueda eficaz, fichas de producto detalladas, carrito persistente, lista de deseos y un proceso de compra fluido; y, del lado del negocio, se requiere un panel de administración que permita mantener el catálogo, los precios, los descuentos, las existencias y los usuarios sin intervención técnica. Reproducir esa experiencia exige integrar un conjunto amplio de tecnologías: un *frontend* dinámico, un servicio web que sirva los datos, una base de datos relacional, almacenamiento de imágenes, mecanismos de autenticación y autorización, y una infraestructura donde desplegar y mantener todo ello.

El presente Trabajo Fin de Grado (en adelante, TFG) nace de la voluntad de abordar ese problema de forma realista y completa, aplicando de manera integrada los conocimientos adquiridos a lo largo del ciclo —desarrollo *web* en entorno cliente y en entorno servidor, despliegue de aplicaciones, bases de datos y sistemas— y, al mismo tiempo, de incorporar tecnologías y prácticas profesionales de actualidad que van más allá del temario estricto: arquitecturas desacopladas cliente-servidor, *frameworks* de componentes con detección de cambios basada en *signals*, interoperabilidad entre *frameworks* mediante estándares del navegador, documentación automática de servicios web, infraestructura como código y canalizaciones de integración y entrega continuas.

El nombre del proyecto, **TechUniverse**, refleja su orientación: un «universo» de productos tecnológicos accesible a través de una tienda en línea.

1.2. Descripción general del proyecto

TechUniverse es un sistema *full-stack* compuesto por cuatro subsistemas claramente diferenciados, alojados en un mismo repositorio (estructura de *monorepo*):

- **Tienda pública (frontend Angular).** Aplicación de página única desarrollada con Angular 21 que ofrece la portada con novedades y ofertas, el catálogo paginado, el listado de productos por categoría, el detalle de producto con sus especificaciones técnicas, el buscador, el carrito de la compra, la lista de deseos, el proceso de finalización de compra (simulado) y la autenticación de usuarios (registro e inicio de sesión).
- **Componentes React embebidos.** Subproyecto independiente desarrollado con React 19 y Vite que se compila a un único *bundle* JavaScript y expone dos *Web Components* —un asistente conversacional flotante (*chatbot*) y un interruptor visual día/noche— que la aplicación Angular consume como si fueran etiquetas HTML nativas.
- **Servicio web REST y panel de administración (backend Laravel).** API REST construida con Laravel 12 que expone los recursos del dominio (productos, categorías, especificaciones, carrito, lista de deseos, usuarios) y la autenticación por *tokens* con Sanctum; está documentada con OpenAPI mediante L5-Swagger. El mismo proyecto incluye un panel de administración —implementado con controladores y vistas Blade— para la gestión del catálogo y de los usuarios.

- **Infraestructura y despliegue.** Definición de la infraestructura en Amazon Web Services mediante Terraform (instancias EC2 para *frontend*, *backend*, base de datos y un *bastion host*; grupos de seguridad; zona DNS privada en Route 53; estado remoto en S3) y canalizaciones de CI/CD con GitHub Actions que ejecutan pruebas, aplican la infraestructura, construyen los artefactos y los despliegan con AWS CodeDeploy. Además de uso de DuckDNS para un dominio y poder hacer https

El proyecto ha sido desarrollado por un equipo de dos personas y gestionado con Git/GitHub (ramas *main* y *develop*), un tablero Trello para el seguimiento de tareas y herramientas de diseño como Figma (interfaz) y draw.io (modelo entidad-relación).

1.3. Objetivos

1.3.1. Objetivo general

Diseñar, implementar y desplegar una plataforma de comercio electrónico tecnológico completa, con arquitectura cliente-servidor desacoplada, que cubra tanto la experiencia del cliente final como la administración del negocio, aplicando tecnologías y prácticas representativas del desarrollo web profesional actual y documentando todo el proceso de forma técnica y rigurosa.

1.3.2. Objetivos específicos

1. **Construir una tienda en línea como SPA** con Angular 21, organizada en componentes *standalone*, con enrutado y carga diferida, gestión de estado reactiva mediante *signals*, diseño responsivo y soporte de modo claro/oscuro.
2. **Desarrollar y documentar un servicio web REST** con Laravel 12 que exponga el catálogo y el resto de recursos del dominio, con autenticación por *tokens* (Sanctum), control de acceso por roles, paginación, ordenación, validación de entradas y respuestas normalizadas mediante *API Resources*, todo ello documentado con OpenAPI.
3. **Modelar el dominio en una base de datos relacional** que represente productos, categorías, especificaciones técnicas (relación con datos en la tabla pivote), carrito de la compra (con cantidades y precios congelados) y lista de deseos, con integridad referencial.
4. **Implementar un panel de administración** que permita gestionar productos (con subida de imágenes y especificaciones), categorías, especificaciones y usuarios, con autenticación de administrador y operaciones CRUD con búsqueda y filtrado.
5. **Demostrar interoperabilidad entre frameworks desarrollando dos componentes en React**, empaquetándolos como Web Components estándar e integrándolos en la aplicación Angular, con sincronización del tema entre ambos mundos.
6. **Gestionar el almacenamiento de imágenes** del catálogo en un servicio de objetos compatible con S3 (Cloudflare R2) mediante Flysystem.
7. **Definir la infraestructura como código** con Terraform y **automatizar el ciclo de integración y entrega** con GitHub Actions y AWS CodeDeploy (ejecución de pruebas, construcción de artefactos, aprovisionamiento de servidores y despliegue).
8. **Establecer una estrategia de pruebas** automatizadas para el *backend* (PHPUnit) y el *frontend* (Vitest) integrada en una canalización de integración continua.
9. **Documentar el proyecto** de forma técnica y académica, incluyendo análisis, diseño, implementación, pruebas, conclusiones y la deuda técnica identificada.

2. Planificación y metodología

2.1. Equipo de trabajo y reparto de tareas

El proyecto ha sido desarrollado por un equipo de dos personas: **Iván Ríos Raya** y **Alexander Sánchez Jara**. El reparto de responsabilidades principales ha sido el siguiente:

Ámbito	Responsable principal
Servicio web REST con Laravel (modelos, controladores, recursos, autenticación, documentación OpenAPI, <i>seeders</i>)	Iván Ríos Raya
Panel de administración (controladores Admin, vistas Blade)	Iván Ríos Raya
Infraestructura como código (Terraform), aprovisionamiento de servidores y canalizaciones de CI/CD (GitHub Actions, CodeDeploy)	Iván Ríos Raya
Tienda pública en Angular (componentes, páginas, servicios, enrutado, interceptor, tematización)	Alexander Sánchez Jara
Componentes React (asistente conversacional e interruptor día/noche) e integración como <i>Web Components</i>	Alexander Sánchez Jara
Conexión del <i>frontend</i> con la API, carrito y lista de deseos, paginación	Alexander Sánchez Jara
Pruebas automatizadas	Reparto según subsistema
Documentación, diseño en Figma y modelo entidad-relación	Conjunto

El trabajo se ha coordinado mediante reuniones de seguimiento, un tablero Trello compartido y el flujo de trabajo de ramas de Git descrito más adelante.

2.2. Gestión del código fuente: estrategia de ramas

El repositorio sigue una **estrategia de ramas sencilla**, adecuada a un equipo pequeño:

- ``main`` — rama estable, con el código listo para producción. Solo recibe integraciones desde `develop` cuando un incremento está suficientemente probado.
- ``develop`` — rama de integración y pruebas, donde confluye el trabajo en curso.

2.3. Herramientas de gestión, diseño y desarrollo

Categoría	Herramienta	Uso en el proyecto
Control de versiones	Git + GitHub	Repositorio <i>monorepo</i> , ramas <i>main/develop</i> , historial, <i>issues</i> implícitos
Integración y entrega continuas	GitHub Actions	Ejecución de pruebas y análisis de estilo; aplicación de Terraform; construcción y despliegue de artefactos con CodeDeploy
Gestión de tareas	Trello	Tablero Kanban del equipo
Diseño de interfaz	Figma	<i>Mockups</i> y prototipo visual de la tienda
Modelado de datos	draw.io	Diagrama entidad-relación
Infraestructura como código	Terraform	Definición de la infraestructura en AWS
Despliegue	AWS CodeDeploy	Despliegue de los artefactos en las instancias EC2
Editor / IDE	Visual Studio Code	Desarrollo (con tareas y configuración de depuración del proyecto)
Backend	PHP 8.2/8.3, Composer, Laravel 12, Artisan	Servicio web REST y panel de administración
Frontend	Node.js 22, npm, Angular CLI 21, Vite	Tienda pública (Angular) y componentes React
Base de datos	MySQL 8	Persistencia del dominio
Almacenamiento de objetos	Cloudflare R2 (compatible con S3)	Imágenes de los productos
Documentación de API	L5-Swagger / OpenAPI 3.0, interfaz Swagger UI	Documentación interactiva de la API
Alojamiento provisional de la API	AlwaysData	Publicación de la API mientras se consolida AWS
Servidor web (servidores AWS)	Apache 2 (con <code>mod_php</code> y <code>mod_proxy</code>)	Servir el <i>frontend</i> estático y la API; <i>reverse proxy</i>
Certificados TLS	Certbot / Let's Encrypt	HTTPS en el servidor del <i>frontend</i>
DNS dinámico	DuckDNS	Nombre de dominio estable pese a IP pública cambiante

2.4. Cronograma y fases del proyecto

El desarrollo se ha extendido aproximadamente desde **noviembre de 2025 hasta mayo de 2026**.

Fase	Período aproximado	Hitos principales
F1. Arranque y prototipos	Noviembre de 2025	Creación del repositorio; <i>mockups</i> en Figma; primeros prototipos de la tienda (HTML/CSS/JS y React); enlace al diseño en el README
F2. Primer <i>backend</i> y exploración de despliegue	Diciembre de 2025	Primera versión del <i>backend</i> ; pruebas iniciales de contenedores y de Terraform para AWS
F3. API y prototipo de <i>chatbot</i>	Enero de 2026	Subida del proyecto de la API; primer prototipo del <i>chatbot</i> ; <i>frontend</i> en React
F4. Consolidación de la API y renombrado del proyecto	Marzo de 2026	Renombrado a «TechUniverse»; subida de la API a un alojamiento; actualización del README
F5. Migración del <i>frontend</i> a Angular y conexión con la API	Abril de 2026 (1.ª mitad)	Migración del <i>frontend</i> a Angular; <i>widget</i> de <i>chat</i> ; conexión a la API real; versiones 0.9 y 1.0 de la API; registro, inicio y cierre de sesión

F6. Componentes React embebidos y funcionalidades de tienda	Abril de 2026 (2.ª mitad)	Integración del componente React en Angular; interruptor de tema; enrutado, paginación; carrito mediante el <i>endpoint</i> de cesta; lista de deseos
F7. Pruebas, panel de administración e infraestructura	Mayo de 2026 (1.ª semana)	Canalizaciones de pruebas; mejoras de la API y del panel de administración; infraestructura Terraform; primeros intentos de CodeDeploy
F8. Despliegue y ajustes finales	Mayo de 2026	Depuración del despliegue (servidor web, <i>reverse proxy</i> , certificados TLS, DNS dinámico, agente de CodeDeploy); ofertas y novedades; ajustes de la lista de deseos; sincronización del <i>chatbot</i> con el tema; redacción de la memoria

3. Tecnologías y estado del arte

3.1. Arquitectura de referencia: SPA + API REST

El proyecto adopta la arquitectura hoy predominante para aplicaciones web ricas: una **aplicación de página única (SPA)** en el cliente, que se comunica con un **servicio web REST** independiente mediante HTTP/JSON. Las características y ventajas de este modelo, frente al modelo tradicional de páginas renderizadas en el servidor, son:

- **Desacoplamiento.** El *frontend* y el *backend* evolucionan de forma independiente y podrían sustituirse o reutilizarse por separado (la misma API podría alimentar una aplicación móvil, por ejemplo).
- **Experiencia de usuario fluida.** Tras la carga inicial, la navegación entre vistas no recarga la página: el cliente actualiza el DOM y solicita al servidor únicamente los datos necesarios, lo que reduce la latencia percibida.
- **Separación de responsabilidades clara.** El *backend* concentra la lógica de negocio, la persistencia y la seguridad; el *frontend*, la presentación y la interacción.
- **Escalabilidad y despliegue independientes.** El *frontend* puede servirse como contenido estático desde un servidor web o una red de distribución de contenidos; el *backend*, escalarse según su carga.

Como contrapartidas, este modelo introduce complejidad adicional: gestión de la autenticación mediante *tokens*, manejo de CORS, necesidad de documentar y versionar la API, y duplicidad de modelos de datos (uno en el servidor y otro en el cliente). El proyecto aborda estas cuestiones con Laravel Sanctum, un *proxy* de desarrollo y un *reverse proxy* en producción, OpenAPI/Swagger y *API Resources* en el servidor combinados con interfaces y *mappers* en el cliente.

A esta arquitectura base se le añade un matiz: dos piezas de interfaz (el *chatbot* y el interruptor día/noche) están escritas en React y se integran en la SPA de Angular mediante *Web Components*, lo que constituye un patrón de *micro-frontend* ligero y un ejercicio de interoperabilidad entre *frameworks*.

3.2. Tecnologías del backend

3.2.1. PHP y Laravel 12

El backend está desarrollado en PHP 8.2 (en los servidores se usa PHP 8.3) utilizando Laravel 12 (versión 12.56). Laravel es un framework muy extendido en el ecosistema PHP y facilita bastante el desarrollo de APIs. Entre lo más relevante que aporta al proyecto están:

- Un sistema de rutas claro y fácil de organizar, con middleware para controlar el acceso.
- Un contenedor de servicios con inyección de dependencias.
- El ORM Eloquent para trabajar con la base de datos mediante modelos.
- Migraciones y seeders para versionar el esquema y generar datos de prueba.
- API Resources para devolver respuestas JSON consistentes.
- Artisan, la herramienta de consola para generar código, ejecutar migraciones, limpiar cachés o lanzar pruebas.

3.2.2. L5-Swagger / OpenAPI

La API está documentada con L5-Swagger (versión 8.6), que integra swagger-php y Swagger UI dentro de Laravel. A partir de anotaciones en los controladores (@OA\...) se genera automáticamente una especificación OpenAPI 3.0 y una interfaz visual accesible en /api/documentation.

La documentación muestra, para cada endpoint, el método HTTP, la ruta, los parámetros, los cuerpos de petición, los esquemas de los recursos y los posibles códigos de respuesta. También incluye el esquema de autenticación mediante token bearer. Contar con documentación OpenAPI facilita el trabajo del frontend y de cualquier servicio externo que necesite consumir la API

3.2.3. Panel de administración: Blade

El panel de administración está construido con controladores propios y vistas Blade, el motor de plantillas de Laravel. Utiliza autenticación basada en sesión (guard web) y un middleware que limita el acceso a usuarios con rol de administrador. Para el diseño se usa Tailwind CSS cargado desde CDN. Este panel permite gestionar los recursos principales del sistema de forma sencilla y sirve como base para futuras ampliaciones.

3.2.4. PHPUnit

Las pruebas automatizadas se desarrollan con PHPUnit (versión 11.5). Se dividen en pruebas unitarias y pruebas de funcionalidad.

Las pruebas de funcionalidad ejecutan peticiones HTTP simuladas contra la API utilizando:

- Una base de datos SQLite en memoria que se reinicia en cada test (RefreshDatabase).
- Autenticación simulada con Sanctum.
- Un sistema de archivos falso para probar subidas de imágenes sin afectar al almacenamiento real.

3.3. Tecnologías del frontend

3.3.1. Angular 21

La tienda pública está desarrollada con **Angular 21** (versión 21.1), una de las versiones más recientes del *framework* de Google. El proyecto aprovecha las capacidades modernas de Angular:

- **Componentes standalone.** No se utilizan `NgModule`: cada componente declara directamente sus dependencias (`imports`), y la aplicación se arranca con `bootstrapApplication`.
- **Signals.** El estado se gestiona con signals (`signal`, `computed`, `effect`), el nuevo modelo de reactividad de Angular, más sencillo y eficiente que `Observable` para el estado local y compartido.
- **Nuevo control flow en plantillas.** Se usa `@if`, `@for` y `@else` en lugar de las directivas estructurales `ngIf/ngFor`.
- **Carga diferida por ruta.** Cada página se carga con `loadComponent()`, generando un chunk JavaScript independiente.

- **Interceptores funcionales.** El *token* de autenticación se añade a las peticiones mediante un `HttpInterceptorFn`.
- **Interoperabilidad RxJS** ↔ `signals` con `toSignal()` para reaccionar a los parámetros de ruta.
- **Esquema CUSTOM_ELEMENTS_SCHEMA** para consumir los *Web Components* de React.

El proyecto utiliza el nuevo *builder* de Angular basado en `esbuild/Vite`, TypeScript en modo estricto y *alias* de rutas de importación. No se emplea renderizado del lado del servidor (SSR).

3.3.2. Tailwind CSS y sistema de diseño

La interfaz se estiliza con **Tailwind CSS** (versión 3 en el *frontend* Angular; versión 4 configurada en los recursos del *backend*), un *framework* de utilidades CSS que permite construir interfaces directamente desde el marcado mediante clases atómicas. Sobre Tailwind se ha definido un **sistema de diseño** propio: un conjunto de **variables CSS** para los colores semánticos (fondo, texto, tarjeta, primario, secundario, *muted*, *accent*, destructivo, borde...), declaradas en dos versiones una para el tema claro y otra para el tema oscuro y mapeadas en la configuración de Tailwind a clases como `bg-card`, `text-foreground` o `bg-primary`. Las tipografías (Inter para el cuerpo y Aldrich para los titulares) se cargan desde Google Fonts.

3.3.3. Lucide y otras librerías de UI

La iconografía proviene de **lucide-angular**, una librería que expone los iconos del conjunto Lucide como componentes Angular; cada componente importa únicamente los iconos que necesita, lo que favorece el *tree-shaking*. El sistema de notificaciones (*toasts*) es **propio** (un servicio y un componente desarrollados a medida, con animaciones de Angular, cuatro tipos: éxito, error, aviso, información, autocierre y barra de progreso. Las animaciones de interfaz se apoyan en `@angular/animations` y en utilidades CSS.

3.3.4. Vitest

Las pruebas del *frontend* se ejecutan con **Vitest** (versión 4), a través del nuevo *runner* de pruebas unificado de Angular, con `jsdom` como DOM simulado. Se mantiene la API de TestBed de Angular y de **HttpTestingController** para probar servicios y componentes.

3.4. React y Web Components: interoperabilidad de frameworks

Dos componentes de interfaz —el asistente conversacional (*chatbot*) y el interruptor visual día/noche— están desarrollados con **React 19** en un subproyecto independiente, compilado con **Vite 8** en modo *library* a un único fichero JavaScript en formato **IIFE** (*Immediately Invoked Function Expression*), con el CSS inyectado en el propio *bundle* y React incluido (no externalizado). Mediante la librería `@r2wc/react-to-web-component`, esos componentes React se envuelven como **Custom Elements** (Web Components).

Los Web Components son el contrato universal que permite que una pieza escrita en un *framework* se consuma desde otro —o desde una página sin *framework*—, lo que constituye un patrón de *micro-frontend*.

3.5. Almacenamiento de objetos: Cloudflare R2 / Amazon S3

Las imágenes de los productos no se guardan en la base de datos ni en el servidor donde corre la aplicación. En su lugar, se almacenan en un servicio de **almacenamiento de objetos compatible con S3**, en este caso **Cloudflare R2**.

Laravel accede a este servicio mediante Fylesystem y el adaptador de S3, configurando un *disk* llamado **R2** con sus credenciales, bucket, endpoint y URL pública.

Cuando se crea o actualiza un producto, la imagen se sube con visibilidad pública y se guarda su URL completa. Si el producto se elimina, también se borra la imagen del almacenamiento.

Usar un servicio externo para los archivos estáticos es habitual porque:

- Evita mezclar imágenes con el código o la base de datos.
- Facilita el despliegue, ya que los servidores pueden ser efímeros.
- Permite servir las imágenes desde una red rápida y distribuida.

3.6. Infraestructura y despliegue

3.6.1. Amazon Web Services (EC2, VPC, Route 53)

La infraestructura del proyecto se ejecuta en AWS, en la región *us-east-1*.

Se utilizan instancias **EC2** con Ubuntu 24.04 dentro de la VPC por defecto. No se crean recursos de red avanzados: se aprovechan las subredes y la configuración básica ya existente.

El acceso a nivel de red se controla mediante **grupos de seguridad**, de forma que cada servidor sólo recibe el tráfico necesario. Hay una IP Elástica para el Bastión.

Además, **Route 53** aloja una zona DNS privada para que las instancias puedan resolverse entre sí.

La elección de AWS se debe a que es un proveedor muy extendido y a la disponibilidad del entorno educativo **AWS Academy / Learner Lab**, cuyas credenciales temporales han condicionado parte del diseño.

3.6.2. Terraform (infraestructura como código)

Toda la infraestructura se define con Terraform (CLI 1.7.0).

En los ficheros *.tf* se declaran:

- Los grupos de seguridad y sus reglas.
- Las cuatro instancias EC2 (frontend, backend, base de datos y bastión).
- La zona DNS y sus registros.
- IP Elástica para el Bastión
- La selección automática de la AMI de Ubuntu.

También se usan variables para parámetros como la región, el dominio, el token de DuckDNS o la contraseña de la base de datos.

El estado de Terraform se guarda en un bucket S3 para mantenerlo sincronizado.

Definir la infraestructura como código permite reproducirla fácilmente, versionarla y revisar los cambios antes de aplicarlos

3.6.3. GitHub Actions (integración y entrega continuas)

El proyecto utiliza GitHub Actions para automatizar tareas de integración y despliegue. Hay tres flujos principales:

- **Testing:** ejecuta los test tanto del backend como del frontend.
- **Infraestructura:** ejecuta terraform init/validate/plan/apply.
- **Despliegue completo:** aplica la infraestructura, construye el backend y el frontend, sube los artefactos a S3 y crea los despliegues en CodeDeploy.

Los dos primeros se lanzan manualmente y el de despliegue se ejecuta al hacer cambios en main (también se ejecutó en la rama develop solo para probar que funcionara todo)

3.6.4. AWS CodeDeploy

El despliegue en las instancias EC2 se realiza con CodeDeploy.

Cada instancia tiene instalado el agente, que descarga el paquete desde S3 y ejecuta los pasos definidos en appspec.yml.

Para el backend, el proceso incluye:

- Copiar el código a /var/www/api.
- Instalar dependencias de producción.
- Generar la clave de la aplicación.
- Ejecutar migraciones y seeders.
- Cachear configuración y rutas.
- Reiniciar el servidor web.

Para el frontend, se copia la compilación de Angular a /var/www/frontend y se reinicia Apache.

3.6.5. Apache, Certbot/Let's Encrypt y DuckDNS

En los servidores se utiliza **Apache 2**.

En el backend, Apache sirve la API con PHP 8.3.

En el frontend, Apache sirve la aplicación Angular y actúa como **proxy inverso** hacia el backend para unificar todo bajo un mismo dominio y un único certificado TLS.

El certificado HTTPS se obtiene con **Let's Encrypt** mediante **Certbot**.

Como la IP pública del entorno educativo cambia cada vez que se inicia la instancia, se usa **DuckDNS** para actualizar automáticamente el subdominio techuniverse.duckdns.org desde el propio *user-data* del servidor.

3.7. Justificación de la pila tecnológica

La pila tecnológica seleccionada busca un equilibrio entre **representatividad profesional**, **coherencia interna del proyecto** y **valor práctico**. Tecnologías como **Laravel**, **Angular**, **React**, **AWS**, **Terraform** y los flujos de **CI/CD** son ampliamente utilizadas en entornos reales, por lo que resultan adecuadas para construir una aplicación moderna y mantenible.

4. Análisis del sistema

Este capítulo identifica los actores del sistema y especifica sus requisitos funcionales y no funcionales, los casos de uso principales y las reglas de negocio.

4.1. Identificación de actores

Se distinguen tres actores:

- **Visitante (usuario anónimo).** Persona que accede a la tienda sin haber iniciado sesión. Puede navegar por el catálogo, ver las categorías, las ofertas y las novedades, consultar el detalle de cualquier producto con sus especificaciones, buscar productos y acceder a las páginas de información. No puede añadir productos al carrito ni a la lista de deseos, ni finalizar una compra: al intentarlo se le invita a iniciar sesión.
- **Usuario registrado (cliente).** Visitante que se ha registrado y ha iniciado sesión. Además de todo lo anterior, puede añadir y eliminar productos de su carrito, modificar las cantidades, gestionar su lista de deseos y acceder al proceso de finalización de compra (simulado). Su carrito y su lista de deseos se persisten en el servidor asociados a su cuenta.
- **Administrador.** Usuario con rol de administrador. Accede al panel de administración mediante un inicio de sesión específico y puede gestionar (crear, listar, editar y eliminar) productos —con su imagen y sus especificaciones técnicas—, categorías, especificaciones y usuarios, y consultar estadísticas básicas del catálogo.

4.2. Casos de uso

A continuación se describen, de forma resumida, los casos de uso más relevantes.

CU-01. Explorar el catálogo. *Actor:* visitante o usuario registrado. *Flujo:* el actor accede a la portada o a una categoría; el sistema solicita a la API la página de productos correspondiente (con la ordenación elegida, en su caso) y la muestra; el actor página o cambia la ordenación; el sistema repite la solicitud. *Resultado:* el actor visualiza los productos.

CU-02. Ver el detalle de un producto. *Actor:* visitante o usuario registrado. *Flujo:* el actor pulsa una tarjeta de producto; el sistema solicita a la API el producto por su identificador (con su categoría y sus especificaciones) y muestra la ficha. *Resultado:* el actor ve la información completa del producto.

CU-03. Buscar productos. *Actor:* visitante o usuario registrado. *Flujo:* el actor escribe un texto en el buscador y envía; el sistema recupera productos y filtra por nombre, descripción o categoría; muestra los resultados paginados. *Resultado:* el actor ve los productos coincidentes.

CU-04. Registrarse. *Actor:* visitante. *Flujo:* el actor abre la página de autenticación, selecciona «Crear cuenta», introduce nombre, correo, contraseña y confirmación; el sistema valida (correo único, contraseña mínima, coincidencia) y, si es correcto, crea el usuario con rol cliente y su cesta vacía; informa del éxito e invita a iniciar sesión. *Resultado:* cuenta creada.

CU-05. Iniciar sesión. *Actor:* visitante registrado. *Flujo:* el actor introduce correo y contraseña; el sistema verifica las credenciales y, si son válidas, emite un *token*, lo devuelve junto con los datos

del usuario; el *frontend* guarda el *token* y el usuario, y carga el carrito y la lista de deseos del usuario desde la API. *Resultado*: sesión iniciada.

CU-06. Añadir un producto al carrito. *Actor*: usuario registrado. *Precondición*: sesión iniciada. *Flujo*: el actor pulsa «Añadir al carrito» en una tarjeta o en la ficha; si el producto ya estaba en el carrito, el sistema incrementa su cantidad; si no, lo añade con cantidad 1 y el precio unitario actual; recarga el carrito y muestra una notificación. *Excepción*: si el actor no ha iniciado sesión, el sistema muestra una notificación informativa y lo redirige a la página de autenticación. *Resultado*: producto en el carrito.

CU-07. Gestionar el carrito. *Actor*: usuario registrado. *Flujo*: el actor abre el carrito; el sistema muestra las líneas, los importes (subtotal, envío, IVA, total); el actor modifica cantidades o elimina líneas; el sistema actualiza el servidor y la vista. *Resultado*: carrito actualizado.

CU-08. Gestionar la lista de deseos. *Actor*: usuario registrado. *Flujo*: el actor pulsa el icono de favorito en una tarjeta; si el producto no estaba, el sistema lo añade a su lista de deseos; si estaba, lo elimina; muestra una notificación. *Excepción*: sin sesión, redirige a autenticación. *Resultado*: lista de deseos actualizada.

CU-09. Finalizar la compra (simulada). *Actor*: usuario registrado con carrito no vacío. *Flujo*: el actor pulsa «Proceder al pago»; el sistema muestra el formulario de *checkout*; el actor lo rellena; el sistema valida; al confirmar, muestra un mensaje de éxito y vacía el carrito. *Resultado*: compra simulada completada.

CU-10. Administrar el catálogo. *Actor*: administrador. *Precondición*: sesión de administrador. *Flujo*: el actor accede al panel; lista productos (puede buscar y filtrar por categoría); crea uno nuevo (nombre, precio, descripción, categoría, existencias, descuento, imagen, especificaciones), lo edita o lo elimina; el sistema valida, almacena la imagen en el servicio de objetos y actualiza la base de datos. *Resultado*: catálogo actualizado.

CU-11. Administrar usuarios, categorías y especificaciones. *Actor*: administrador. *Flujo*: análogo a CU-10 para cada recurso, con sus reglas (no eliminar la propia cuenta; no eliminar categorías con productos). *Resultado*: datos actualizados.

CU-12. Consultar la documentación de la API. *Actor*: desarrollador (del *frontend* o tercero). *Flujo*: accede a `/api/documentation`; el sistema muestra la interfaz Swagger con todos los *endpoints*, parámetros, esquemas y respuestas. *Resultado*: documentación consultada.

4.3. Reglas de negocio

- **RN-1. Roles.** Existen dos roles de usuario: usuario (cliente) y admin. El registro público y la creación de usuarios desde la API crean siempre clientes; solo el panel de administración o los *seeders* crean administradores. El administrador tiene acceso a todos los recursos de cliente (jerarquía de roles).
- **RN-2. Descuento y precio.** Cada producto tiene un porcentaje de descuento (0–100). El precio con descuento se calcula en el servidor como $\text{precio} - \text{precio} \times \text{descuento} \div 100$. Un producto «está en oferta» si su descuento es mayor que cero.
- **RN-3. Existencias.** Cada producto tiene unas existencias (entero ≥ 0 , por defecto 0). En la ficha se avisa de «existencias bajas» cuando son menores que 10. No se permite añadir al carrito un producto sin existencias desde la ficha de detalle (validación del cliente).

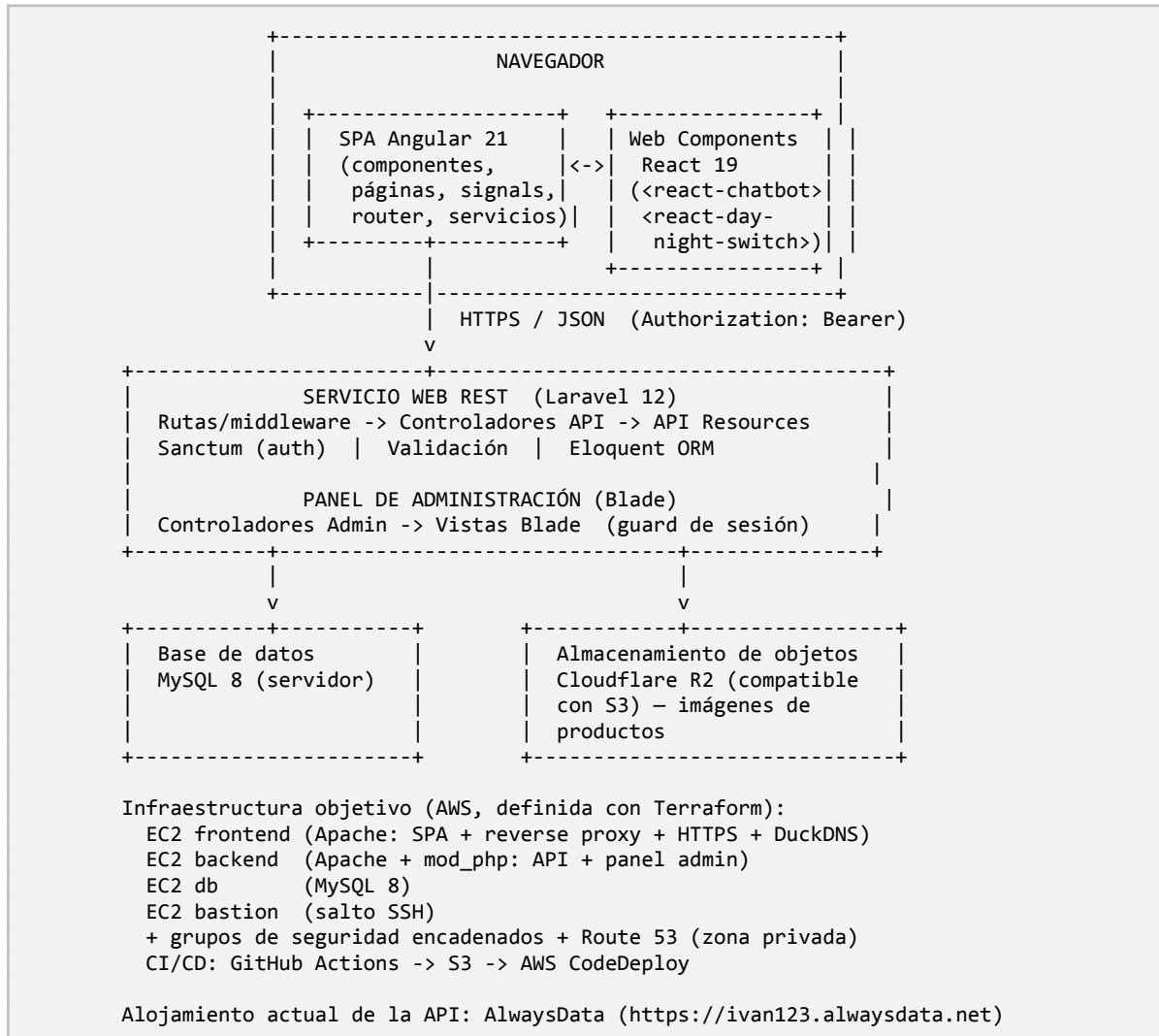
- **RN-4. Unicidad en la lista de deseos.** Un producto no puede aparecer dos veces en la lista de deseos de un mismo usuario.
- **RN-5. Carrito.** Cada usuario tiene exactamente una cesta. Al añadir un producto que ya está en la cesta se incrementa la cantidad; el precio unitario de la línea se fija al añadir el producto y no varía aunque cambie el precio del producto.
- **RN-6. Importes del carrito.** El IVA aplicado es del 21 % sobre el subtotal. Los gastos de envío son gratuitos si el subtotal supera los 50 € y de 5,99 € en caso contrario. *(Estas reglas se aplican en el cliente; el servidor no calcula impuestos ni envío.)*
- **RN-7. Integridad referencial.** Al eliminar una categoría se eliminan en cascada sus productos; al eliminar un producto o un usuario se eliminan en cascada sus líneas de carrito, sus entradas de lista de deseos y sus asociaciones de especificación; al eliminar una especificación se eliminan sus asociaciones con productos. No obstante, desde el panel de administración no se permite eliminar una categoría que tenga productos asociados (regla adicional de la interfaz).
- **RN-8. Tokens.** Los tokens de acceso emitidos por Sanctum no caducan automáticamente; se invalidan al cerrar sesión.

5. Diseño del sistema

Este capítulo describe el diseño de la solución: la arquitectura general, el modelo de datos, el diseño de la API REST, del *frontend*, de la interfaz de usuario, del panel de administración, de la infraestructura y de la seguridad.

5.1. Arquitectura general

La arquitectura es **cliente-servidor desacoplada**, organizada en capas:



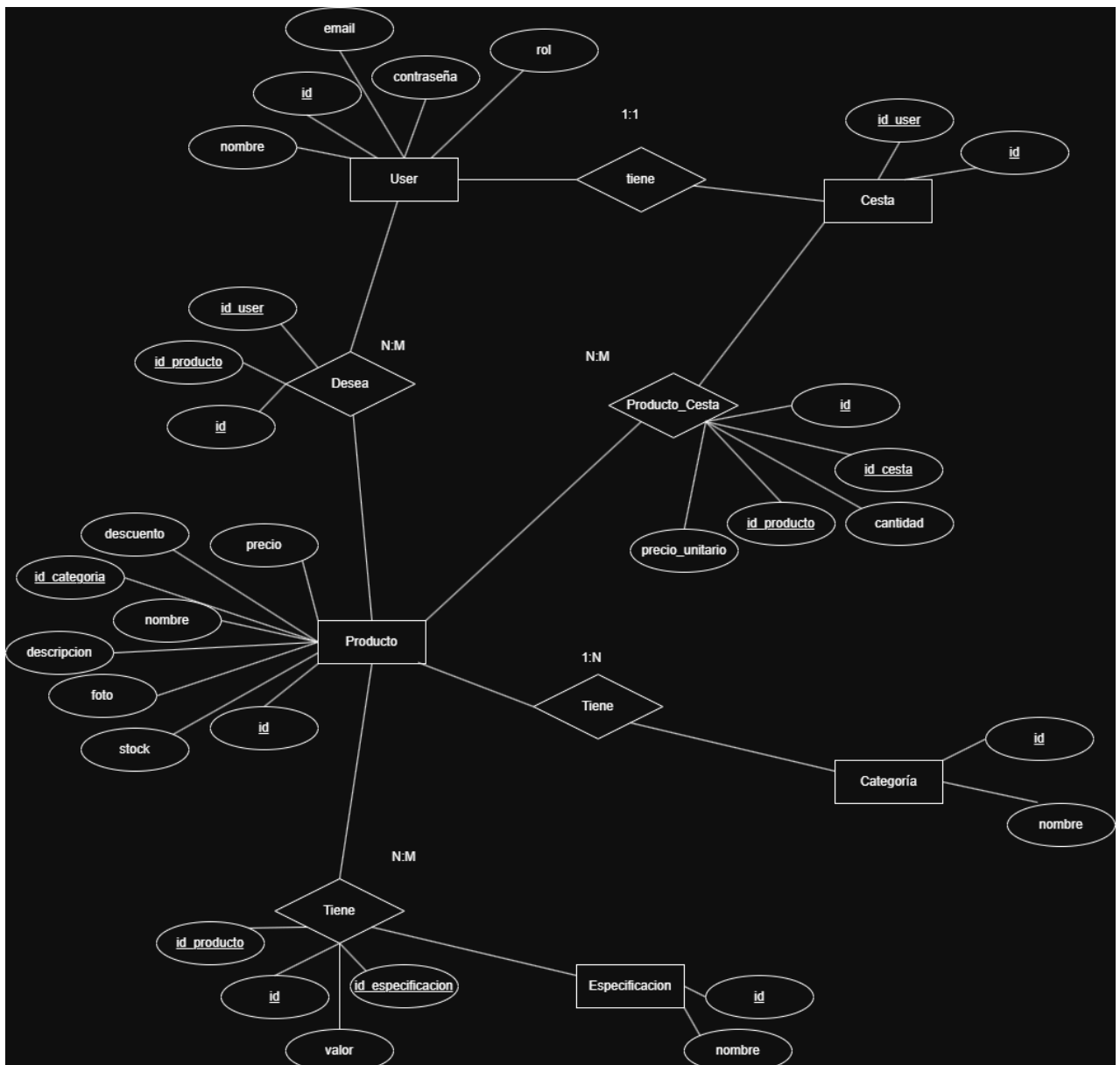
Los componentes son:

- **Cliente.** Una SPA en Angular que renderiza la interfaz, gestiona el estado con *signals*, enruta entre vistas y se comunica con la API por HTTP. Dentro de ella, dos *Web Components* React (el *chatbot* y el interruptor día/noche) actúan como *widgets* embebidos; la comunicación Angular → React es por *binding* de propiedad.
- **Servicio web REST.** Una API en Laravel que aplica las reglas de negocio, persiste los datos con Eloquent, gestiona la autenticación con Sanctum y devuelve respuestas JSON normalizadas con *API Resources*. El mismo proyecto Laravel sirve también el **panel de administración** mediante controladores y vistas Blade con autenticación de sesión.

- **Persistencia.** Una base de datos relacional (MySQL en el servidor; SQLite en desarrollo y pruebas) y un servicio de almacenamiento de objetos (Cloudflare R2) para las imágenes.
- **Infraestructura.** En producción objetivo, cuatro instancias EC2 (con los roles de *frontend*, *backend*, base de datos y *bastion*), aprovisionadas con Terraform y desplegadas con CodeDeploy; mientras tanto, la API se aloja en AlwaysData.

5.2. Modelo Entidad/Relación

El dominio se modela con las siguientes entidades y relaciones ([Modelo E/R](#)):



El modelo se compone de 5 Entidades (Producto, Categoría, Especificación, Cesta y Usuario) y 5 Relaciones, de las cuales 3 de ellas son muchos a muchos de los cuales salen otras 3 tablas.

5.3. Diseño de la API REST

La API sigue las convenciones REST sobre HTTP/JSON, con el prefijo `/api`. Sus principios de diseño son:

- **Recursos.** Los recursos del dominio son productos, categorías, especificaciones, asociaciones producto-especificación, la cesta, las líneas de la cesta, la lista de deseos y los usuarios; además del subsistema de autenticación.
- **Verbos HTTP.** GET para lecturas, POST para creaciones, PUT para actualizaciones y DELETE para borrados.
- **Autenticación y autorización.** Los *endpoints* de lectura del catálogo son públicos. Los de cliente (cesta, lista de deseos) requieren un *token* válido y el rol usuario (el rol admin también pasa). Los de administración requieren *token* y rol admin. La protección se implementa con el *middleware* `auth:sanctum` y un *middleware* de rol parametrizable.
- **Paginación.** Los listados de productos (catálogo completo, productos en oferta y productos por categoría) se pagan a 12 elementos, devolviendo la estructura estándar de Laravel (`data`, `links`, `meta`). El resto de listados se devuelven completos.
- **Ordenación y filtrado.** El listado de productos admite el parámetro `?sort` con los valores `precio_asc`, `precio_desc`, `novedad_asc` y `novedad_desc`.
- **Respuestas normalizadas.** Cada recurso se transforma con un *API Resource* que define los campos expuestos (con nombres en español), incorpora datos calculados (precio con descuento, subtotales, totales del carrito) y resuelve las URL de las imágenes.
- **Códigos de estado.** Se devuelven los códigos adecuados: 200/201 en éxito, 401 si falta autenticación, 403 si falta permiso, 404 si el recurso no existe, 422 si la validación falla (con los mensajes de error).
- **Documentación.** Todo lo anterior se documenta con anotaciones OpenAPI y se publica con Swagger UI en `/api/documentation`.

5.4. Diseño del frontend

El *frontend* Angular se diseña con la siguiente estructura:

- **Componente raíz (App).** Contiene la barra de navegación, el contenedor de rutas (`<router-outlet>`), el contenedor de notificaciones y el *Web Component* del *chatbot* (presente en todas las páginas).
- **Configuración (app.config.ts).** Registra los proveedores de la aplicación: el enrutador (con *binding* de entradas de componente), las animaciones (carga diferida), el cliente HTTP con el interceptor de autenticación y el escuchador global de errores del navegador.
- **Enrutado (app.routes.ts).** Nueve rutas, todas con carga diferida (`loadComponent`): portada, categoría (`category/:category`), detalle de producto (`product/:id`), carrito, *checkout*, autenticación, búsqueda (`search?q=`), lista de deseos y comodín 404. Cada ruta define su título de página. No hay guardas de ruta: la protección de las rutas «privadas» se realiza de forma imperativa en los componentes y servicios.
- **Componentes reutilizables (components/).** Barra de navegación (con buscador, contadores reactivos de carrito y lista de deseos, menú de categorías y conmutador de tema), tarjeta de producto, paginación (anterior/siguiente), conmutador de tema (componente alternativo no utilizado) y notificación *toast*.

- **Páginas (pages/).** Portada, categoría, detalle de producto, búsqueda, carrito, *checkout*, lista de deseos, autenticación y 404.
- **Servicios (services/).** Servicios *singleton* para autenticación (con *BehaviorSubject* del usuario actual), productos (acceso a la API, con *mappers* de los DTO de la API al modelo de cliente), carrito (estado con *signals*, sincronizado con la API), lista de deseos (ídem), tema (estado con *signal*, persistido) y notificaciones (cola con *signal*).
- **Modelos (models/).** Interfaces TypeScript que normalizan los datos: producto, usuario, ítem de carrito e ítem de lista de deseos.
- **Interceptor (interceptors/).** Interceptor funcional que añade la cabecera *Authorization: Bearer <token>* a las peticiones cuando hay sesión.

La **integración con la API** usa la URL base relativa */api*: en desarrollo, un *proxy* de Angular (*proxy.conf.json*) la redirige al *backend* real (*AlwaysData*), evitando problemas de CORS; en producción, el servidor web del *frontend* hace de *reverse proxy* hacia el *backend*. El interceptor inyecta el *token*; el manejo de errores HTTP es puntual en cada servicio (notificación al usuario o silencioso).

5.5. Diseño de la interfaz de usuario

La interfaz se ha diseñado previamente en **Figma** y se ha implementado con **Tailwind CSS** sobre un sistema de diseño propio:

- **Identidad visual.** Paleta con un degradado primario azul → violeta (*gradient-primary*) presente en botones, *badges* y el *hero* de la portada; tipografías Inter (cuerpo) y Aldrich (titulares); esquinas redondeadas, sombras suaves y microinteracciones (hover con escala/sombra, animaciones de entrada).
- **Modo claro/oscuro.** Dos juegos de variables CSS (HSL) para los colores semánticos; el tema se aplica con una clase en el elemento `<html>` y se persiste en *localStorage*, respetando además la preferencia del sistema operativo (*prefers-color-scheme*). El conmutador de la barra de navegación es un *Web Component* React (el interruptor visual día/noche) con un botón Angular transparente superpuesto que ejecuta el cambio.
- **Diseño responsivo.** Contenedores fluidos; la barra de navegación oculta el buscador y la barra de categorías en pantallas pequeñas; las cuadrículas de productos pasan de una a cuatro columnas según el ancho; las páginas con resumen lateral (carrito, *checkout*) lo colocan en una columna *sticky* en pantallas anchas.
- **Componentes de interfaz.** Tarjeta de producto (con *badges* de «Nuevo» y descuento, botón de favorito que aparece al pasar el ratón y botón de añadir al carrito), paginación anterior/siguiente, notificaciones *toast* (con icono y color por tipo, autocierre y barra de progreso), barra de navegación con contadores reactivos y *hero* de la portada.
- **Iconografía.** Iconos del conjunto Lucide, importados individualmente por componente.
- **Componentes embebidos.** El *chatbot* flotante (botón circular en la esquina inferior derecha que despliega una ventana de conversación) y el interruptor día/noche (animación de sol a luna y de nubes a estrellas).

5.6. Diseño del panel de administración

El panel de administración se sirve desde el mismo proyecto Laravel, bajo el prefijo `/admin`, con autenticación basada en sesión (*guard web*) y un *middleware* que solo permite el acceso a usuarios con rol `admin` (de lo contrario, redirige al inicio de sesión del panel). Su diseño contempla:

- **Inicio de sesión específico** (`/admin/login`): valida las credenciales, comprueba el rol y regenera la sesión.
- **Cuadro de mando** (`/admin`): muestra cuatro estadísticas (productos, usuarios, categorías, productos sin existencias) y los productos más recientes añadidos.
- **CRUD de recursos** (recursos: productos, categorías, especificaciones, usuarios) con listados paginados, búsqueda y, en productos, filtro por categoría. El formulario de producto incluye la subida de imagen (al servicio de objetos R2) y un repetidor para gestionar sus especificaciones técnicas (parejas especificación–valor). Reglas adicionales: no se puede eliminar una categoría con productos asociados ni eliminar la propia cuenta de usuario.
- **Disposición**: un *layout* con barra lateral (cuadro de mando y cada recurso), datos del usuario, cierre de sesión y zona de mensajes; estilizado con Tailwind servido por CDN.

5.7. Diseño de la infraestructura de despliegue

La infraestructura objetivo, definida con Terraform, consta de **cuatro instancias EC2** `t2.medium` con Ubuntu 24.04 LTS sobre la VPC por defecto, con los roles:

- **bastion** — punto de salto SSH; acepta SSH (puerto 22) desde cualquier IP.
- **frontend** — *sirve el SPA Angular con Apache, actúa de reverse proxy hacia el backend (rutas `/api`, `/admin`, `/sanctum`, `/storage`), termina TLS (certificado de Let's Encrypt) y mantiene actualizado el subdominio DuckDNS; acepta HTTP (80) y HTTPS (443) desde Internet y SSH desde el bastion. Es el único punto de entrada de tráfico de usuario*.*
- **backend** — *sirve la API y el panel de administración con Apache + `mod_php` 8.3; acepta HTTP (80) solo desde el frontend y SSH desde el bastion*. Su fichero `.env` apunta a la base de datos por su IP privada.*
- **db** — *ejecuta MySQL 8; acepta MySQL (3306) y SSH solo desde el backend*.*

El control de acceso se realiza con **grupos de seguridad encadenados** (cada servidor solo recibe el tráfico imprescindible y, salvo el *frontend* y el *bastion*, únicamente desde otros servidores). **Route 53** aloja una **zona DNS privada** (`techuniverse.com`) con registros internos (`bastion.`, `frontend.`, `backend.`) hacia las IP privadas. El **estado de Terraform** se guarda en un *bucket* de **S3**. El aprovisionamiento de cada instancia se realiza con plantillas de *user-data* (`cloud-init`) que instalan y configuran el servidor web, PHP, Composer, MySQL, los *virtual hosts*, Certbot y el agente de CodeDeploy según el rol.

El **flujo de despliegue** diseñado (canalización `codedeploy.yml` de GitHub Actions) encadena: aplicación de la infraestructura con Terraform (recreando las instancias de *frontend* y *backend* para que se vuelva a ejecutar su *user-data*), construcción de los artefactos (`composer install --no-dev` para el *backend*; `npm ci && npm run build` para el *frontend*), empaquetado en `.zip`, subida a S3 y creación de despliegues en AWS CodeDeploy, que en cada instancia ejecuta los *hooks* del `appspec.yml` (copia del código, instalación de dependencias, migraciones, cacheo y

reinicio del servidor web). Las particularidades y el estado real de este diseño se detallan en la implementación.

5.8. Diseño de la seguridad

El diseño de seguridad se articula en varias capas:

- **Autenticación.** La API usa *tokens* de acceso personales de Sanctum (cabecera `Authorization: Bearer`); el panel de administración usa autenticación basada en sesión (*guard web*). Las contraseñas se almacenan con *hash* (Bcrypt, mediante el *cast hashed*). El cierre de sesión revoca el *token* actual.
- **Autorización.** Las rutas se protegen con `auth:sanctum` y con un *middleware* de rol parametrizable: las de cliente exigen rol usuario (el admin también pasa); las de administración (API y panel), rol admin.
- **Validación de entradas.** Todas las creaciones y actualizaciones validan los datos en el servidor (tipos, longitudes, formatos, unicidad de correo, tipo y tamaño de las imágenes), devolviendo 422 con los mensajes en caso de error.
- **Transporte.** En producción, la comunicación es por HTTPS (certificado de Let's Encrypt en el servidor de *frontend*, que termina TLS y propaga X-Forwarded-Proto al *backend*).
- **Red.** El acceso a la base de datos está restringido por grupo de seguridad al servidor de *backend*; el *backend* solo recibe tráfico del *frontend*; el SSH interno solo desde el *bastion*.
- **CORS.** En desarrollo se evita mediante el *proxy* de Angular; en producción, sirviendo todo bajo el mismo dominio gracias al *reverse proxy*.
- **Almacenamiento de ficheros.** Las imágenes se guardan en un servicio de objetos con visibilidad pública controlada; los servidores de aplicaciones son sin estado.

Se reconocen, no obstante, varias **debilidades** propias de un proyecto de aprendizaje, que se recogen en la deuda técnica: gestión de secretos en texto plano (en el estado de Terraform, en el *user-data* y en el `.env`), uso del superusuario de MySQL como usuario de aplicación, SSH del *bastion* abierto a todo Internet, instancias «privadas» con IP pública (al no haber subredes privadas ni NAT), *tokens* sin caducidad y reinicio destructivo de la base de datos en cada despliegue.

6. Implementación

Este capítulo describe la implementación de cada subsistema: la organización del repositorio, el servicio web REST con Laravel, el panel de administración, el *frontend* Angular, los componentes React embebidos y la infraestructura y el despliegue. Se incluyen fragmentos de código ilustrativos.

6.1. Organización del repositorio

El proyecto se aloja en un único repositorio Git (estructura de *monorepo*) con la siguiente organización de carpetas de primer nivel:

```
Tech-Universe/
├── .github/workflows/      # Canalizaciones de CI/CD (GitHub Actions)
├── Angular-frontend/      # Tienda pública (SPA Angular 21)
├── React-components/      # Subproyecto React (chatbot + interruptor día/noche)
├── techuniverse-api/      # Servicio web REST + panel de administración (Laravel 12)
├── despliegue-aws/        # Infraestructura como código (Terraform)
├── documentacion/         # Documentación necesaria como esta memoria o la presentación
└── README.md              # Documentación general y enlaces del proyecto
```

Las ramas son *main* (estable) y *develop* (integración). El *bundle* compilado de los componentes React (*react-components.iife.js*) se versiona en *Angular-frontend/public/* (se copia manualmente desde la salida de Vite); el resto de artefactos de compilación (carpetas *node_modules/*, *vendor/*, *dist/*, *.angular/*) se ignoran. El README enlaza la documentación de la API, el repositorio original de la API, el diseño en Figma, el modelo entidad-relación en draw.io y el tablero Trello.

6.2. Implementación del backend (API REST con Laravel)

6.2.1. Estructura del proyecto Laravel

El proyecto *techuniverse-api/* sigue la estructura estándar de Laravel 12. El arranque se centraliza en *bootstrap/app.php*, donde se configuran los grupos de rutas (*api*, *web*), se confía en los *proxies* (*trustProxies(at: '*')*, necesario tras el *reverse proxy* de producción) y se registran los **alias de middleware**:

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->trustProxies(at: '*');
    $middleware->alias([
        'rol'      => \App\Http\Middleware\CheckRol::class,          // usado en la API
        'is_admin' => \App\Http\Middleware\IsAdmin::class,          // registrado, sin uso
        'admin'    => \App\Http\Middleware\AdminMiddleware::class,  // usado en el panel
    ]);
    $middleware->redirectGuestsTo('/admin/login');
});
```

La configuración relevante incluye: *config/auth.php* (*guard* por defecto *web*, *provider* *users* sobre *App\Models\User*), *config/sanctum.php* (*expiration* a *null* — los *tokens* no caducan), *config/database.php* (conexión por defecto *SQLite*; configuración completa para *MySQL* con *utf8mb4*), *config/filesystems.php* (un *disk* *r2* de tipo *s3* con las variables *R2_**, además del *disk*

s3 clásico) y `config/15-swagger.php` (ruta de la documentación `api/documentation`, escaneo de anotaciones en `app/`). En desarrollo, `composer dev` arranca de forma concurrente el servidor de Artisan, el escuchador de la cola, `pail` (registro en vivo) y `npm run dev`.

6.2.2. Modelos Eloquent y relaciones

Los modelos del dominio están en `app/Models/`. El modelo `User` (sobre la tabla `users`) usa los *traits* `HasApiTokens` (`Sanctum`), `HasFactory` y `Notifiable`, oculta `password` y `remember_token`, castea `password` a `hashed` y declara las relaciones con la lista de deseos y con la cesta:

```
class User extends Authenticatable {
    use HasApiTokens, HasFactory, Notifiable;
    protected $fillable = ['name', 'email', 'password', 'rol'];
    protected $hidden = ['password', 'remember_token'];
    protected function casts(): array {
        return ['email_verified_at' => 'datetime', 'password' => 'hashed'];
    }
    public function deseos() { return $this->belongsToMany(Producto::class, 'producto_user'); }
    public function cesta() { return $this->hasOne(Cesta::class); }
}
```

El modelo `Producto` declara `$fillable` (`nombre`, `precio`, `stock`, `descripcion`, `descuento`, `categoria_id`, `foto`), un atributo calculado `foto_url` (un accesor que devuelve la URL absoluta de la imagen —tolerando tanto rutas relativas del disco `r2` como URL ya absolutas— o una imagen por defecto) y sus relaciones:

```
class Producto extends Model {
    use HasFactory;
    protected $fillable = ['nombre', 'precio', 'stock', 'descripcion', 'descuento', 'categoria_id', 'foto'];
    protected $appends = ['foto_url'];
    public function getFotoUrlAttribute() { /* resuelve URL absoluta a R2 o imagen por defecto */ }

    public function categoria() { return $this->belongsTo(Categoria::class); }
    public function usuariosQueLoDesean() { return $this->belongsToMany(User::class, 'producto_user'); }
    public function cestas() { return $this->belongsToMany(Cesta::class, 'producto_cestas', 'producto_id', 'cesta_id'); }
    public function especificaciones() { return $this->belongsToMany(Especificaciones::class, 'producto_especificacions', 'producto_id', 'especificacion_id')->withPivot('valor')->withTimestamps(); }
    public function productoEspecificaciones() { return $this->hasMany(ProductoEspecificacion::class); }
}
```

`Categoria` declara `hasMany(Producto::class)`. La entidad `especificaciones` (nótese el nombre en minúscula y plural, una incorrección de convención que se documenta en la deuda técnica) declara la relación inversa muchos a muchos con `Producto`. Las entidades pivote enriquecidas `ProductoEspecificacion` (atributo `valor`) y `ProductoCesta` (atributos `cantidad` y `precio_unitario`) se modelan como modelos propios con sus `belongsToMany`. `Cesta` declara `belongsTo(User::class)` y `belongsToMany(Producto::class, 'producto_cestas', ...)->withPivot('precio_unitario', 'cantidad')`. Existen dos modelos huérfanos sin uso (`Usuario` sobre la tabla `usuarios`, y `ProductoCesta`), recogidos en la deuda técnica.

6.2.3. Migraciones y poblado de datos

El esquema se versiona con **migraciones** en database/migrations/: usuarios, categorías, productos (con categoria_id como clave foránea con borrado en cascada, precio DECIMAL(10,2), stock INT por defecto 0, descuento DECIMAL(5,2) por defecto 0, foto con una URL por defecto), *tokens* de acceso personales (Sanctum), la tabla pivote producto_user (con índice único (user_id, producto_id) y borrados en cascada), cestas (user_id con borrado en cascada), la tabla pivote producto_cestas (cantidad, precio_unitario DECIMAL(8,2), borrados en cascada), especificaciones y la tabla pivote producto_especificacions (valor, borrados en cascada); además de una migración que añade la columna published_at a productos (sin uso posterior). Para poblar la base de datos en desarrollo y pruebas hay **factories** (CategoriaFactory, ProductoFactory, EspecificacionFactory, UserFactory) y **seeders** (CategoriaSeeder, ProductoSeeder, EspecificacionSeeder, ProductoEspecificacionSeeder, DeseosSeeder, UsersSeeder), orquestados por DatabaseSeeder. El UsersSeeder crea un administrador (admin@techstore.com) y un cliente (usuario@techstore.com).

6.2.4. Controladores de la API y recursos

Los controladores de la API están en app/Http/Controllers/Api/. Las rutas se definen en routes/api.php, agrupadas por recurso y *middleware*. Un extracto representativo:

```
// Auth (públicas)
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);
Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/user', [AuthController::class, 'me']);
});

// Productos (lectura pública; escritura solo admin)
Route::get('/productos/count', [ProductoController::class, 'count']);
Route::get('/productos/oferta', [ProductoController::class, 'oferta']);
Route::get('/productos', [ProductoController::class, 'index']); //
?sort=precio_asc|precio_desc|novedad_asc|novedad_desc
Route::get('/productos/{id}', [ProductoController::class, 'show']);
Route::middleware(['auth:sanctum', 'rol:admin'])->group(function () {
    Route::post('/productos', [ProductoController::class, 'store']);
    Route::match(['put', 'patch'], '/productos/{id}', [ProductoController::class, 'update']);
    Route::delete('/productos/{id}', [ProductoController::class, 'destroy']);
});

// Cesta y líneas de la cesta (rol usuario)
Route::middleware(['auth:sanctum', 'rol:usuario'])->group(function () {
    Route::get('/cesta', [CestaController::class, 'index']);
    Route::post('/cesta/productos', [ProductoCestaController::class, 'store']);
    Route::put('/cesta/productos/{id}', [ProductoCestaController::class, 'update']);
    Route::delete('/cesta/productos/{id}', [ProductoCestaController::class, 'destroy']);
});
```

Los controladores aplican la lógica de negocio: ProductoController@index ordena según ?sort y pagina a 12, con carga ansiosa de la categoría y de las especificaciones; ProductoController@oferta filtra por descuento > 0; ProductoController@store/update validan la entrada (nombre, precio numérico mínimo 0,01, descripcion, categoria_id, foto de tipo imagen y tamaño máximo) y gestionan la subida (y el borrado de la imagen anterior) en el disco r2; CategoriaController@show devuelve la categoría con sus productos paginados; AuthController@register crea el usuario con rol usuario y su cesta vacía;

AuthController@login autentica con Auth::attempt y emite el *token* con createToken('auth_token')->plainTextToken, devolviendo access_token, token_type, user y una redirección; AuthController@logout ejecuta currentAccessToken()->delete(); ProductoCestaController@store añade el producto a la cesta del usuario (incrementando la cantidad si ya estaba, o creando la línea con el precio_unitario actual); WishlistController@store usa syncWithoutDetaching para no duplicar; UserController es un apiResource para administración. La tabla completa de *endpoints* está en el Anexo A.

Las respuestas se normalizan con ***API Resources*** (app/Http/Resources/): ProductoResource expone id, nombre, precio, descuento, precioDescuento (calculado), stock, descripcion, modificado, foto (URL absoluta), categoria ({nombre}) y especificaciones (lista de {nombre, valor}); CategoriaResource y CategoriaConProductosResource; EspecificacionResource y EspecificacionConProductosResource; ProductoEspecificacionResource; CestaResource (con usuario, productos —cada uno con precio_unitario, cantidad, subtotal—, precio_total y cantidad_total); UserResource (con los campos renombrados al castellano: nombre, correo, rol); y WishlistResource (con desea cargado condicionalmente con whenLoaded).

6.2.5. Autenticación y autorización

La autenticación de la API usa **Sanctum**: el modelo User incorpora HasApiTokens; las rutas protegidas usan el *middleware* auth:sanctum; el *token* se transmite en Authorization: Bearer <token>. La autorización por roles se implementa con el *middleware* CheckRol, parametrizado con el rol requerido:

```
class CheckRol {
    public function handle(Request $request, Closure $next, string $rol) {
        $user = $request->user();
        if (!$user) return response()->json(['mensaje' => 'No autenticado'], 401);
        if ($user->rol === 'admin' || $user->rol === $rol) return $next($request);
        return response()->json(['mensaje' => 'No tienes permisos para acceder aquí'], 403);
    }
}
```

Es decir, un administrador supera siempre la comprobación (jerarquía de roles). El panel de administración usa, en cambio, autenticación basada en sesión: AdminAuthController@login ejecuta Auth::attempt, comprueba que el rol sea admin (si no, hace Auth::logout() y devuelve un error), regenera la sesión y redirige al cuadro de mando; el *middleware* AdminMiddleware protege el grupo de rutas del panel y redirige a /admin/login a quien no esté autenticado o no sea administrador. El proyecto contiene además dos *middlewares* de rol adicionales (CheckRole, IsAdmin) sin uso efectivo, y un fichero TrustProxies.php residual, recogidos en la deuda técnica.

6.2.6. Almacenamiento de imágenes en R2

Las imágenes de los productos se gestionan con **Flysystem** sobre el disco r2 (tipo s3, configurado en config/filesystems.php con R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET, R2_ENDPOINT y R2_PUBLIC_URL). Al crear un producto, la imagen recibida (multipart/form-data) se sube con storePublicly('productos', 'r2') y se almacena su URL absoluta en la columna foto; al actualizar, si llega una nueva imagen, se borra la anterior (si era una ruta relativa del disco) y se sube la nueva; al eliminar el producto, se borra también el objeto. El accesor foto_url del modelo resuelve la URL final para las respuestas. El SDK de AWS para PHP y

league/flysystem-aws-s3-v3 proporcionan el soporte. *(Se ha detectado una pequeña inconsistencia: hay tres puntos del código —el accesor del modelo, `ProductoResource` y CategoriaConProductosResource— que construyen la URL de la imagen de formas ligeramente distintas; conviene centralizarlo. Queda recogido en la deuda técnica.)*

6.2.7. Documentación de la API con OpenAPI

La API se documenta con **L5-Swagger**. El controlador base (app/Http/Controllers/Controller.php). Cada controlador de la API anota sus métodos con @OA\Get/Post/Put/Delete (ruta, etiquetas, parámetros, cuerpo de la petición, respuestas) y define @OA\Schema para los recursos. La especificación generada se guarda en storage/api-docs/api-docs.json y la interfaz interactiva se publica en /api/documentation; en producción está disponible en el alojamiento de AlwaysData. *(Existen algunos desajustes entre las rutas documentadas en las anotaciones y las rutas reales —por ejemplo, plural frente a singular en categorías—, también recogidos en la deuda técnica.)*

6.3. Implementación del panel de administración

El panel de administración operativo está construido con **controladores propios** (app/Http/Controllers/Admin/) y **vistas Blade** (resources/views/admin/). Las rutas son: el inicio de sesión del panel (/admin/login, GET y POST, sin *middleware*) y un grupo protegido con ['auth','admin'] y prefijo admin que incluye el cuadro de mando (/admin → AdminDashboardController@index), Route::resource para productos, categorías, especificaciones y usuarios y el cierre de sesión (POST /admin/logout).

- **AdminDashboardController@index** calcula las estadísticas (total_productos, total_usuarios, total_categorias, sin_stock) y los cinco productos más recientes, y renderiza admin.dashboard.
- **AdminProductoController** ofrece el CRUD completo y filtro por categoría (paginate(15)->withQueryString()), gestiona la subida de la imagen al disco r2, el borrado de la imagen anterior y un repetidor de especificaciones (especificaciones[*][especificacion_id] y [valor]) que crea, borra y recrea filas en producto_especificacions.
- **AdminCategoriaController** ofrece el CRUD con búsqueda y withCount('productos'), e impide eliminar categorías con productos asociados.
- **AdminEspecificacionController** ofrece el CRUD con búsqueda y withCount('productos').
- **AdminUserController** ofrece el CRUD con búsqueda (nombre/correo) y filtro por rol; valida la confirmación de contraseña y el rol (in:admin,usuario); al crear un usuario también crea su cesta; impide eliminar la propia cuenta.

Las vistas Blade se organizan con un *layout* (admin/layouts/app.blade.php) con barra lateral (cuadro de mando, productos, categorías, usuarios, especificaciones), datos del usuario, cierre de sesión y zona de mensajes *flash*, estilizado con Tailwind servido por CDN; y plantillas index, create y edit para cada recurso, con tablas (imagen, categoría, precio, *badge* de descuento, existencias resaltadas si son 0) y formularios (multipart/form-data, previsualización de la imagen, repetidor de especificaciones), incluyendo la pantalla de inicio de sesión (admin/auth/login.blade.php).

6.4. Implementación del frontend (Angular)

6.4.1. Arranque y configuración

El proyecto `Angular-frontend/` (Angular 21.1) arranca en `src/main.ts` con `bootstrapApplication(App, appConfig)`. La configuración (`src/app/app.config.ts`) registra los proveedores:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
    provideAnimationsAsync(),
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
};
```

El componente raíz `App` declara `imports: [RouterOutlet, Navbar, Toast]` y `schemas: [CUSTOM_ELEMENTS_SCHEMA]` (para consumir el *Web Component* `<react-chatbot>`), y su plantilla coloca la barra de navegación, el `<router-outlet>`, el contenedor de *toasts* y el *chatbot* (presente en todas las rutas). El proyecto usa el *builder* `@angular/build:application`, TypeScript en modo estricto, *alias*es de importación (`@services/*`, `@models/*`, `@components/*`, etc.) y carga las tipografías desde Google Fonts en `src/styles.css`, donde también se definen las variables CSS del tema.

6.4.2. Enrutado y carga diferida

El enrutado (`src/app/app.routes.ts`) define nueve rutas, **todas con carga diferida** mediante `loadComponent()`, lo que genera un *chunk* JavaScript por página.

No hay guardas de ruta: las rutas «privadas» (`cart`, `wishlist`, `checkout`) son accesibles, pero los servicios solo cargan datos si hay sesión, y las acciones que requieren sesión (añadir al carrito o a la lista de deseos, proceder al pago) comprueban el *token* y redirigen a `/auth` si falta. El mapa completo de rutas está en el Anexo C.

6.4.3. Componentes reutilizables

- **Navbar** Contiene el logo (SVG), el buscador, el switch de tema (el *Web Component*), el botón de usuario/sesión muestra el saludo y el cierre de sesión; si no, un enlace a `/auth`), y los botones de lista de deseos y carrito con **contadores reactivos**, más una barra de categorías con enlaces directos. Los contadores son reactivos porque proceden de *signals/computed* de los servicios.
- **ProductCard** Recibe ocho `@Input()` (`id`, `name`, `price`, `originalPrice?`, `image`, `category`, `isNew?`, `discount?`). Su plantilla muestra los *badges* de «Nuevo» y descuento, el botón de favorito (corazón) que aparece al pasar el ratón o si el producto está en la lista de deseos, la imagen enlazada a `/product/:id`, el nombre, la categoría, el precio (y el original tachado) y el botón «Añadir al carrito». Los métodos `handleAddToCart` y `toggleWishlist` comprueban si hay *token*: si no, muestran una notificación y redirigen a `/auth`.
- **Pagination**. Recibe `currentPage` y `lastPage`, emite `pageChange`. Solo se renderiza si `lastPage > 1`; ofrece botones «Anterior»/«Siguiente» (deshabilitados en los extremos) y el indicador «Página X de Y».

- **Toast.** Inyecta `ToastService` y usa animaciones de Angular (`@angular/animations`) para la entrada (desde abajo) y la salida (escalando). Renderiza las notificaciones: icono y color según el tipo (éxito, error, aviso, información).

6.4.4. Páginas

- **Home.** Portada: *hero* de presentación con dos llamadas a la acción, sección de características, sección de novedades y sección de ofertas, ambas como cuadrículas de `<app-product-card>`.
- **Category.** Listado de una categoría. A partir del *slug* de la ruta deriva el título y el modo.
- **ProductDetail.** Ficha de producto. Obtiene el producto por su *id*, muestra la imagen (con zoom al pasar el ratón), los *badges*, el precio (y el original), el aviso de existencias bajas (si `stock < 10`), la descripción, dos tarjetas de servicio y la tabla de especificaciones.
- **Search.** Resultados de búsqueda. A partir del parámetro de consulta `?q=`, obtiene y filtra en el cliente por nombre, descripción o categoría; pagina los resultados en bloques de 12. *(Es una búsqueda del lado del cliente sobre la primera página del catálogo)*
- **Carrito.** Muestra las líneas con imagen, nombre, precio unitario, precio descuento, controles de cantidad, borrado y un resumen lateral con subtotal, envío, IVA y total. El botón «Proceder al pago» comprueba la sesión y navega a `/checkout`.
- **Checkout.** Formulario de finalización de compra. Al confirmar, si el formulario es válido, muestra una notificación de éxito, vacía el carrito (solo en el cliente) y navega a la portada; es un proceso **simulado** (no crea pedido ni cobra).
- **Wishlist.** Muestra los productos guardados como cuadrícula de `<app-product-card>`, o un estado vacío.
- **Auth.** Una sola vista con pestañas y dos formularios reactivos. `onLogin()` llama a `authService.login(...)` o `onRegister()` comprueba que la contraseña y su confirmación coincidan y llama a `authService.register(...)`
- **NotFound.** Página 404 con un «404» decorativo, mensaje, botones a la portada y a la búsqueda, y enlaces a categorías populares.

6.4.5. Servicios y gestión de estado con señales

Todos los servicios son `@Injectable({ providedIn: 'root' })`. El estado compartido se modela con *signals*:

- **AuthService.** Mantiene `currentUserSubject (BehaviorSubject<User | null>)`, expuesto como `currentUser$`. Persiste el *token* (`localStorage.auth_token`) y el usuario (`localStorage.currentUser`). En el constructor, `checkToken()` rehidrata el usuario si hay *token*. Métodos: `getToken()`, `login(credentials)` (POST `/api/login`; guarda el *token* y el usuario), `register(userData)` (POST `/api/register`), `logout()` (POST `/api/logout`; en cualquier caso, `clearSession()` borra el *token* y el usuario y emite `null`).
- **ProductService.** Sin estado; inyecta `HttpClient`. Métodos: `getProducts(page)` (GET `/api/productos?page=`), `getCategorias()` (GET `/api/categoria`), `getProductsByCategory(id)` (GET `/api/categoria/{id}`), `getProductById(id)` (GET `/api/productos/{id}`), `getNewProducts(limit)` (GET `/api/productos?sort=novedad_asc`), `getOfferProducts()` (GET `/api/productos/oferta`). Los *mappers* privados normalizan los

DTO de la API (campos en español, especificaciones, descuento, precio con descuento) al modelo de cliente y la respuesta paginada (data, meta).

- **CartService.** En el constructor se suscribe a `authService.currentUser$`: si hay usuario, `loadCart()` (GET /api/cesta); si no, vacía el *signal*. Métodos: `addItem(...)` (incrementa la cantidad si ya está, o POST /api/cesta/productos y recarga), `updateQuantity(...)` (PUT /api/cesta/productos/{id}), `removeItem(...)` (DELETE /api/cesta/productos/{id}).
- **WishlistService.** En el constructor se suscribe a `currentUser$`: si hay usuario, `loadWishlist(user.id)` (GET /api/deseos/{userId}); si no, vacía. Métodos: `isInWishlist(id)`, `addToWishlist(...)` (POST /api/deseos), `removeFromWishlist(id)` (DELETE /api/deseos/{id} con `user_id` en el cuerpo).
- **ThemeService.** En el constructor (si es navegador) registra un *effect* que aplica el tema (clase en `<html>`) y lo persiste en `localStorage.theme`. La inicialización prioriza `localStorage` → `prefers-color-scheme` → `dark`. Métodos: `toggleTheme()`, `setTheme(theme)`.
- **ToastService.** Métodos: `success/error/info/warning(message, title?)` que delegan en `addToast`, el cual genera un identificador, añade la notificación y programa su eliminación a los 3 segundos.

Ejemplo ilustrativo (CartService):

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private http = inject(HttpClient);
  private authService = inject(AuthService);
  private toastService = inject(ToastService);

  private itemsSignal = signal<CartItem[]>([]);
  readonly items = this.itemsSignal.asReadonly();
  readonly totalItems = computed(() => this.items().reduce((s, i) => s + i.quantity, 0));
  readonly subtotal = computed(() => this.items().reduce((s, i) => s + i.price * i.quantity, 0));

  constructor() {
    this.authService.currentUser$.subscribe(user => {
      if (user) this.loadCart(); else this.itemsSignal.set([]);
    });
  }
  // loadCart() -> GET /api/cesta ; addItem() -> POST /api/cesta/productos ; ...
}
```

6.4.6. Consumo de la API e interceptor de autenticación

Todos los servicios usan la URL base relativa /api. En desarrollo, `proxy.conf.json` redirige /api al *backend* real (<https://ivan123.alwaysdata.net>), con `changeOrigin: true`, evitando CORS; en producción, el servidor web del *frontend* hace de *reverse proxy*. El **interceptor de autenticación** es funcional y añade la cabecera `Authorization: Bearer <token>` a todas las peticiones cuando hay sesión:

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getToken();
  if (token) req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  return next(req);
};
```

6.5. Implementación de los componentes React (Web Components)

6.5.1. El subproyecto React-components

El subproyecto `React-components/` usa **React 19.2** y **ReactDOM 19** (con `react-dom/client`), **TypeScript 5.9**, **Vite 8** y `@r2wc/react-to-web-component` 2.0.3`.

6.5.2. Empaquetado y registro como Custom Elements

`vite.config.ts` configura una compilación en modo *library*: punto de entrada `src/register.tsx`, nombre global `ReactWebComponents`, formato ``iife``, fichero de salida `react-components.iife.js`; el CSS se inyecta en el propio JavaScript y **React no se externaliza** (va incluido en el *bundle*), de modo que el componente es autónomo y no depende de la página anfitriona. El punto de entrada `register.tsx` envuelve cada componente React como *Custom Element* y lo registra:

```
import r2wc from '@r2wc/react-to-web-component';
import ChatBot from './components/Chatbot';
import DayNightSwitch from './components/DayNightSwitch';
import './components/ChatBot.css';
import './components/DayNightSwitch.css';

const ChatBotWC = r2wc(ChatBot);
const DayNightSwitchWC = r2wc(DayNightSwitch, { props: { theme: 'string' } });

if (!customElements.get('react-chatbot')) customElements.define('react-chatbot', ChatBotWC);
if (!customElements.get('react-day-night-switch')) customElements.define('react-day-night-switch', DayNightSwitchWC);
```

`@r2wc/react-to-web-component` crea, por debajo, una subclase de `HTMLElement` que, al insertarse en el DOM (`connectedCallback`), monta el componente React con `createRoot(elemento).render(...)`; cuando cambia un atributo observado (en el caso del interruptor, `theme`, declarado con `{ props: { theme: 'string' } }`), vuelve a renderizar con la nueva propiedad.

6.5.3. El asistente conversacional (chatbot)

`Chatbot.jsx` implementa un **asistente conversacional con respuestas predefinidas basadas en coincidencia de palabras clave**: no usa ningún servicio externo ni modelo de lenguaje. Mantiene una base de conocimiento, un *array* de objetos `{ keywords, answer }` con cinco intenciones (productos/catálogo, envío a domicilio, horario, saludo, agradecimiento) y, ante un mensaje del usuario, devuelve la respuesta de la primera intención cuya palabra clave esté contenida en el mensaje o un mensaje genérico de reserva. La interfaz consta de un botón flotante que despliega una ventana de conversación con cabecera, lista de mensajes, indicador de «escribiendo...» y campo de entrada. El estado se gestiona con `useState` (`isOpen`, `inputValue`, `isLoading`, `messages`) y `useRef`; hay efectos para auto-desplazar la conversación y para cerrar la ventana con la tecla *Escape*. Al enviar un mensaje, se simula una latencia de 800 ms antes de mostrar la respuesta. El componente se adapta al modo oscuro mediante reglas CSS condicionadas a `html.dark`.

6.6. Implementación de la infraestructura y el despliegue

6.6.1. Infraestructura como código con Terraform

El módulo `despliegue-aws/` define la infraestructura en AWS (proveedor `hashicorp/aws` 6.x, región `us-east-1`, **estado remoto en un bucket de S3 —`techuniverse-terraform-state`—, sin bloqueo de estado*). Mediante `data sources` obtiene la AMI más reciente de `Ubuntu 24.04 LTS (Canonical)` y la VPC por defecto* de la cuenta. Declara:

- **Cinco grupos de seguridad** (`all`, `Bastion`, `FrontEnd`, `BackEnd`, `db`) en la VPC por defecto, definidos «vacíos» y con las reglas en recursos aparte: una regla de **salida** total (`all` → `0.0.0.0/0`, todos los protocolos) adjunta a todas las instancias; y **ocho reglas de entrada**: SSH (22) al *bastion* desde Internet; SSH (22) a *frontend* y a *backend* solo desde el grupo del *bastion*; SSH (22) a *db* solo desde el grupo de *backend*; HTTP (80) al *frontend* desde Internet; HTTP (80) al *backend* solo desde el grupo del *frontend*; HTTPS (443) al *frontend* desde Internet; y MySQL (3306) a *db* solo desde el grupo de *backend*.
- **Cuatro instancias EC2** `t2.medium` con la AMI de `Ubuntu 24.04`: *Bastion* (sin perfil de instancia; solo SSH), *db* (con `user-data db.tftpl`), *BackEnd* (con perfil de instancia `LabInstanceProfile` —el del laboratorio de AWS Academy—; `user-data backend.tftpl` con la IP privada de *db*), *FrontEnd* (con perfil `LabInstanceProfile`; `user-data frontend.tftpl` con la IP privada de *BackEnd* y el `token` de `DuckDNS`). Todas con `user_data_replace_on_change = true` (un cambio en el `user-data` destruye y recrea la instancia) y con la cadena de dependencias `db` → `BackEnd` → `FrontEnd`.
- **Una zona DNS privada** en `Route 53` (`techuniverse.com`, asociada a la VPC por defecto) con tres registros A (*bastion.*, *frontend.*, *backend.*) hacia las IP privadas.
- **Una IP Elástica**, la cual está asociada a la IP Pública de la Instancia Bastión.

Las **variables** de entrada son: `key_name` (par de claves EC2, obligatoria), `region` (def. `us-east-1`), `domain` (def. `techuniverse.com`), `duckdns_token` (sensible, obligatoria), `Instance_Profile` (def. `LabInstanceProfile`), `Instance_Type` (def. `t2.medium`) y `db_root_password` (sensible, obligatoria). No hay outputs. Como se observa, **no se crean** la VPC ni su red (subredes, *internet gateway*, NAT, tablas de rutas), ni RDS, ni un *bucket* de S3, ni roles IAM, ni balanceador, ni grupo de escalado: se reutilizan los recursos de la VPC por defecto y el perfil de instancia del laboratorio. Estas decisiones —y sus implicaciones de seguridad— se recogen en la deuda técnica.

6.6.2. Aprovisionamiento de los servidores (user-data)

Cada instancia se aprovisiona con una plantilla de `user-data` (`cloud-init`) renderizada por Terraform:

- **backend.tftpl** (recibe la IP privada de *db* y la contraseña de la base de datos): instala `Apache 2`, `PHP 8.3` y sus extensiones (`mysql`, `xml`, `mbstring`, `curl`, `zip`, `bcmath`, `intl`, `libapache2-mod-php8.3`), `Composer` y `Ruby` (requisito del agente de `CodeDeploy`); habilita los módulos de `Apache` `rewrite`, `ssl` y `headers`; escribe un *virtual host* con `DocumentRoot /var/www/api/public` y `AllowOverride All`; deja un `.env` base en `/var/www/.env.base` con `APP_ENV=production`, `APP_URL/ASSET_URL` apuntando al dominio público, `DB_HOST` con la IP privada de *db* y `DB_PASSWORD` con la contraseña; y, finalmente, instala y arranca el **agente de CodeDeploy**. No clona repositorios ni ejecuta migraciones (eso lo hace `CodeDeploy`).

- **frontend.tftpl** (recibe la IP privada de BackEnd y el *token* de DuckDNS): instala Apache 2 (sin PHP) y habilita los módulos `rewrite`, `ssl`, `headers`, `proxy`, `proxy_http` y `proxy_wstunnel`; actualiza el subdominio **DuckDNS** (`techuniverse6.duckdns.org`) con la IP pública actual; instala **Certbot** (vía *snap*); escribe un *virtual host* HTTP (para la validación del certificado); espera 30 s; si no existe el certificado, ejecuta `certbot --apache -d techuniverse6.duckdns.org --non-interactive --agree-tos` (con un correo de contacto), de forma idempotente y *fail-safe* (si falla, no rompe Apache); escribe el *virtual host* HTTPS definitivo con ProxyPass/ProxyPassReverse de `/api`, `/admin`, `/sanctum` y `/storage` hacia la IP privada del *backend*, propagación de `X-Forwarded-Proto: https` y una `RewriteRule` que devuelve `index.html` para cualquier ruta que no sea un archivo existente ni empiece por esas rutas (fallback del SPA); y, finalmente, instala y arranca el agente de CodeDeploy.
- **db.tftpl** (recibe la contraseña de la base de datos): instala `mysql-server` (MySQL 8); cambia el `bind-address` a `0.0.0.0` (para aceptar conexiones del *backend*; el control de acceso lo hace el grupo de seguridad); arranca MySQL; y ejecuta el SQL de inicialización: pone contraseña a `root@localhost` (con `mysql_native_password`), crea la base de datos `techuniverse` (`utf8mb4/utf8mb4_unicode_ci`), crea el usuario `root@'%'` con la misma contraseña y le concede todos los privilegios sobre `techuniverse`. (El uso del superusuario `'root'` como usuario de aplicación es una debilidad recogida en la deuda técnica.)

6.6.3. Integración y entrega continuas con GitHub Actions

Hay tres flujos de trabajo en `.github/workflows/`:

- **ci-cd.yml («CI/CD Pipeline»)**. Se ejecuta **solo manualmente** (`workflow_dispatch`). Un único *job* con un servicio MySQL 8.0 de apoyo: clona el repositorio; para el *backend* configura PHP 8.2 con Composer, ejecuta `composer install`, el análisis de estilo `./vendor/bin/pint --test`, genera un `.env.testing` apuntando al MySQL del servicio, `php artisan key:generate --env=testing`, `php artisan migrate --env=testing --force` y `php artisan test --env=testing`; para el *frontend* configura Node 22, `npm ci` y `npm test`. No usa secretos de AWS. Es la **puerta de calidad** (no despliega nada).
- **terraformdeploy.yml («Terraform AWS Deploy»)**. Se ejecuta **solo manualmente**. Configura las credenciales temporales de AWS (incluido `AWS_SESSION_TOKEN`) y las variables `TF_VAR_*` desde los secretos del repositorio; ejecuta `terraform init`, `validate`, `plan` y `apply -auto-approve`. Crea o actualiza **solo la infraestructura**.
- **codedeploy.yml («Despliegue del código»)**. Se ejecuta al hacer *push* a `main` con cambios en `techuniverse-api/**`, `Angular-frontend/**` o `React-components/**`, y también manualmente. Tiene tres *jobs*: `terraform` (igual que el anterior); y, dependientes de él, `deploy-backend` (espera a que la instancia BackEnd pase las comprobaciones de estado y aparezca registrada; configura PHP 8.3; `composer install --no-dev --optimize-autoloader`; empaqueta `techuniverse-api` en `backend.zip` excluyendo `.git`, `node_modules` y `.env`; lo sube a `s3://techuniverse-terraform-state/deployments/backend.zip`; y crea el despliegue con `aws deploy create-deployment --application-name techuniverse-backend --deployment-group-name techuniverse-backend-group ...`) y `deploy-frontend` (configura Node 20; `npm ci`; `npm run build`; arma un paquete con la carpeta `browser/` del *build*, el `appspect.yml` y los *scripts*; lo empaqueta en `frontend.zip`; lo sube a S3; y crea el despliegue `techuniverse-frontend`).

Los secretos del repositorio son `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`, `DB_ROOT_PASSWORD`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `AWS_KEY_NAME` y el *token* de DuckDNS. Las aplicaciones y los grupos de despliegue de CodeDeploy (techuniverse-backend/techuniverse-frontend) deben crearse a mano en la consola de AWS (no los crea Terraform).

6.6.4. Despliegue con AWS CodeDeploy

Cada componente tiene un `appspec.yml` (version: 0.0, os: linux):

- **Frontend** (Angular-frontend/appspec.yml): copia la carpeta browser del *build* a `/var/www/frontend`; *hooks*: `BeforeInstall` → `scripts/before_install.sh` (`rm -rf /var/www/frontend/*`), `ApplicationStart` → `scripts/start_server.sh` (`chown -R www-data:www-data /var/www/frontend`; `systemctl restart apache2`).
- **Backend** (techuniverse-api/appspec.yml): copia todo el bundle a `/var/www/api`; *hooks*: `BeforeInstall` → `scripts/before_install.sh` (`apt-get update`), `AfterInstall` → `scripts/after_install.sh` (instala PHP 8.3 si faltara; ajusta permisos; copia el `.env` desde `/var/www/.env.base` si no existe; `composer install --no-dev --optimize-autoloader`; `php artisan key:generate --force`; ``php artisan migrate:fresh --seed --force`` —reinicia y resiembra toda la base de datos en cada despliegue—; `php artisan optimize:clear`, `config:cache`, `route:cache`, `view:cache`; ajusta permisos de storage y `bootstrap/cache`; `systemctl restart apache2`), `ApplicationStart` → `scripts/start_server.sh` (diagnóstico de servicios y `systemctl reload apache2`). El reinicio destructivo de la base de datos en cada despliegue es adecuado para una demostración, pero no para un entorno con datos reales; se recoge en la deuda técnica (la mejora obvia es usar `php artisan migrate --force incremental` y sembrar solo la primera vez).

7. Pruebas

7.1. Estrategia de pruebas

La estrategia de pruebas combina **pruebas automatizadas** (unitarias y de funcionalidad) integradas en una canalización de **integración continua**, con **pruebas manuales** de aceptación a lo largo del desarrollo. El énfasis de la automatización se ha puesto en los **servicios** —donde se concentra la lógica— y en la **API** —el contrato entre *frontend* y *backend*—, dado que son las partes más críticas y las que más se benefician de una red de seguridad ante regresiones. Las pruebas de interfaz de los componentes y páginas Angular, y las pruebas extremo a extremo, han quedado fuera del alcance por restricciones de tiempo (se recogen como mejora).

7.2. Pruebas del backend

El *backend* se prueba con **PHPUnit 11.5**, con dos suites (Unit y Feature) configuradas en `phpunit.xml` para ejecutarse en un entorno testing con **SQLite en memoria** (`DB_DATABASE=:memory:`), caché y colas en memoria, *mailer* en *array* y `BCRYPT_ROUNDS=4` (para acelerar). Las pruebas de funcionalidad usan el *trait* `RefreshDatabase` (migran la base de datos en cada prueba), `Laravel\Sanctum\Sanctum::actingAs($user, ['*'])` para autenticar, y `Storage::fake('r2')` con `UploadedFile::fake()->image(...)` para las subidas de imagen. En total hay alrededor de **36 métodos de prueba** repartidos en doce ficheros, que cubren:

Suite / fichero	Casos cubiertos
AuthApiTest	Registro de usuario (201, fila creada); inicio de sesión (estructura <code>access_token/token_type/user</code>); cierre de sesión autenticado
CategoriaApiTest	Listado; listado con productos; categoría con productos; 404; creación, actualización y borrado (como administrador)
EspecificacionApiTest	Listado; listado con productos; especificación con productos; 404; creación, actualización y borrado (admin)
ProductoApiTest	Listado; detalle; 404; creación con imagen falsa (admin + <code>Storage::fake('r2')</code>); actualización; borrado
ProductoEspecificacionApiTest	Asociar especificación a producto (admin); actualizar el valor; eliminar; 404 al actualizar/eliminar inexistente
CestaApiTest	Consulta de la cesta con un producto añadido; verificación de la estructura y de los totales (<code>precio_total, cantidad_total</code>)
ProductoCestaApiTest	Añadir producto a la cesta (fila en <code>producto_cestas</code> con <code>cantidad</code> y <code>precio_unitario</code>); actualizar cantidad; eliminar
UsuarioApiTest	Listado (admin); detalle; 404; creación, actualización y borrado
WishlistApiTest	Listado (admin); lista de deseos de un usuario; 404; añadir producto (fila en <code>producto_user</code>); eliminar
ExampleTest (Feature/Unit)	Pruebas de ejemplo (GET / devuelve 200; <code>assertTrue(true)</code>)

La cobertura efectiva incluye el CRUD de cada recurso principal, las tablas pivote (cesta, lista de deseos, producto-especificación), el flujo de autenticación y los casos 404. No se prueban explícitamente los códigos 403 de autorización (las rutas protegidas se ejercitan siempre con un administrador, que sí pasa), la mayoría de las validaciones 422, ni la ordenación `?sort` o los

endpoints productos/oferta y productos/count; tampoco hay pruebas del panel Blade. El comando `composer test` limpia la configuración y ejecuta `php artisan test`.

7.3. Pruebas del frontend

El *frontend* se prueba con **Vitest 4** a través del nuevo *runner* unificado de Angular (@angular/build:unit-test), con jsdom como DOM simulado, manteniendo la API de TestBed y de HttpTestingController. Hay seis ficheros de prueba:

Fichero	Casos cubiertos
app.spec.ts	El componente raíz App se crea (con provideRouter, provideHttpClientTesting, provideNoopAnimations y window.matchMedia simulado)
auth.service.spec.ts	Creación; getToken() nulo sin sesión; lectura de un token existente de localStorage; inicio de sesión (POST /api/login: método, cuerpo, y guardado del token y del usuario tras la respuesta); cierre de sesión (POST /api/logout: limpieza)
cart.service.spec.ts	Creación; carrito vacío inicial; al emitir un usuario, GET /api/cesta y mapeo; cálculo de totalItems y subtotal; clearCart(); updateQuantity(_, 0) no hace petición; updateQuantity(5, 3) hace PUT /api/cesta/productos/5; removeItem(7) hace DELETE /api/cesta/productos/7
wishlist.service.spec.ts	Creación; lista vacía inicial; isInWishlist falso en lista vacía; al emitir un usuario, GET /api/deseos/{id} y mapeo; addToWishlist (POST /api/deseos) y actualización; removeFromWishlist (DELETE) y actualización
theme.service.spec.ts	Creación; setTheme('light'/'dark'); toggleTheme() en ambas direcciones (con window.matchMedia y localStorage simulados)
toast.service.spec.ts	Creación; sin notificaciones inicialmente; añadir éxito/error/aviso/información; con título; remove(id); varias notificaciones simultáneas; autocierre a los 3 s (con temporizadores falsos de Vitest)

7.4. Pruebas manuales y de aceptación

A lo largo del desarrollo se han realizado **pruebas manuales** continuas, especialmente al integrar incrementos: registro e inicio de sesión, navegación por el catálogo y las categorías, paginación, ordenación, búsqueda, ficha de producto, añadir/quitar/actualizar el carrito, lista de deseos, proceso de *checkout* simulado, modo claro/oscuro, comportamiento del *chatbot*, panel de administración (CRUD de productos con imagen y especificaciones, categorías, usuarios) y consulta de la documentación Swagger. La fase final del proyecto (despliegue) ha requerido, además, abundantes **pruebas sobre infraestructura real**: arranque de las instancias, ejecución del *user-data*, configuración del servidor web y del *reverse proxy*, emisión del certificado TLS, actualización del DNS dinámico, registro del agente de CodeDeploy y ejecución de los *hooks* de despliegue; estas pruebas se reflejan en la intensa actividad de *commits* de esa fase.

7.5. Integración continua

El flujo de trabajo `ci-cd.yml` de GitHub Actions constituye la canalización de **integración continua**: levanta un servicio MySQL 8, configura PHP y Node, instala dependencias, ejecuta el análisis de estilo del código PHP (Laravel Pint), migra la base de datos de prueba y ejecuta las suites de PHPUnit y de Vitest. Actualmente se dispara de forma manual (`workflow_dispatch`); una mejora inmediata sería activarlo también en *push* y en *pull request* para que actúe como puerta de calidad automática en cada cambio (recogido en la deuda técnica).

8. Conclusiones y líneas futuras

8.1. Grado de consecución de los objetivos

El proyecto ha cubierto el **objetivo general** —diseñar, implementar y desplegar una plataforma de comercio electrónico tecnológico completa con arquitectura cliente-servidor desacoplada— y la práctica totalidad de los **objetivos específicos**:

Objetivo	Estado	Observaciones
1. Tienda en línea como SPA con Angular 21	Conseguido	Componentes <i>standalone</i> , <i>signals</i> , carga diferida, modo claro/oscuro, diseño responsivo
2. Servicio web REST con Laravel documentado con OpenAPI	Conseguido	~32 <i>endpoints</i> , Sanctum, roles, paginación, ordenación, validación, <i>API Resources</i> , Swagger
3. Modelado del dominio en base de datos relacional	Conseguido	Categorías, productos, especificaciones (con valor en la tabla pivote), carrito (cantidad y precio congelado) y lista de deseos, con integridad referencial
4. Panel de administración	Conseguido	Panel Blade operativo (CRUD de productos con imagen y especificaciones, categorías, especificaciones, usuarios)
5. Interoperabilidad entre <i>frameworks</i> (React en Angular)	Conseguido	<i>Chatbot</i> e interruptor día/noche como <i>Web Components</i> ; sincronización del tema
6. Almacenamiento de imágenes en servicio de objetos	Conseguido	Cloudflare R2 (compatible con S3) mediante Flysystem
7. Infraestructura como código y CI/CD	Conseguido	Terraform define la infraestructura; GitHub Actions y CodeDeploy automatizan el ciclo
8. Estrategia de pruebas automatizadas con CI	Conseguido	Pruebas de los servicios del <i>backend</i> (PHPUnit) y del <i>frontend</i> (Vitest) y canalización de CI
9. Documentación técnica y académica	Conseguido	La presente memoria

En conjunto, se considera que el resultado es satisfactorio: una solución *full-stack* funcional, representativa de las prácticas profesionales actuales, con sus limitaciones documentadas con honestidad.

8.2. Dificultades encontradas y lecciones aprendidas

- **Curva de aprendizaje.** Varias tecnologías (Angular 21 con *signals* y el nuevo *control flow*, Laravel Sanctum, L5-Swagger, react-to-web-component, AWS CodeDeploy) se han aprendido durante el propio desarrollo. El enfoque incremental y los prototipos previos han sido clave para asimilarlas sin bloquear el avance.
- **Interoperabilidad Angular ↔ React.** Integrar dos *frameworks* exigió comprender bien el estándar *Web Components* y tomar decisiones de diseño (renunciar al *Shadow DOM* para que el tema global afectara a los componentes React; resolver la interactividad del interruptor con un botón Angular superpuesto). La lección: los estándares de la plataforma son el mejor «pegamento» entre tecnologías.

- **Despliegue en la nube.** Con diferencia, la parte más laboriosa. Las particularidades del entorno de laboratorio (credenciales temporales, perfil de instancia preexistente, ausencia de recursos de red propios), la depuración del *reverse proxy*, los certificados TLS, el DNS dinámico y el agente de CodeDeploy requirieron muchos ciclos de prueba y error sobre infraestructura real. La lección: la infraestructura como código es imprescindible (permite reconstruir el entorno una y otra vez), pero un despliegue robusto exige más de lo que parece (idempotencia, gestión de secretos, alta disponibilidad), y conviene tener un alojamiento alternativo mientras se consolida.
- **Desarrollo en paralelo.** El trabajo simultáneo de dos personas en subsistemas distintos, integrado de forma frecuente, ha funcionado bien, pero ha dejado algunas inconsistencias (modelos, *middlewares* o componentes duplicados; nombres con erratas; documentación desactualizada en algún punto), que se han catalogado para su saneamiento.
- **Alcance.** Mantener el alcance bajo control —marcando como simulados o futuros la pasarela de pago, los pedidos, la búsqueda en servidor o la alta disponibilidad— ha permitido entregar un sistema completo y coherente en el tiempo disponible.

8.3. Valoración final

TechUniverse ha permitido al equipo recorrer, de principio a fin, el ciclo de vida de una aplicación web profesional: análisis y diseño, modelado de datos, desarrollo de un servicio web documentado, desarrollo de una interfaz moderna, interoperabilidad entre tecnologías, almacenamiento en la nube, infraestructura como código, integración y entrega continuas y pruebas automatizadas. Más allá del resultado —una tienda en línea funcional con su panel de administración y su API—, el mayor valor del proyecto ha sido el **aprendizaje integrado**: comprender cómo encajan entre sí tantas piezas, qué decisiones de diseño implican qué compromisos, y por qué prácticas como la infraestructura como código, la documentación de APIs o las canalizaciones de CI/CD son hoy estándares de la industria. La identificación honesta de la deuda técnica y de las limitaciones —especialmente en el despliegue— no resta valor al trabajo: forma parte de la madurez profesional reconocer lo que funciona, lo que es simulado y lo que queda por hacer. El proyecto deja, además, una base sólida y bien estructurada sobre la que abordar las líneas de trabajo futuro.